GIÁO TRÌNH & BÀI GIẢNG CHUYÊN SÂU

POST-TRAINING CHO MÔ HÌNH NGÔN NGỮ LỚN (LLMs):

Cơ sở Toán học, Tối ưu hóa Sở thích, Học Tăng cường Suy luận, Chắt lọc Tri thức và Kiến trúc Hệ thống

Hệ quy chiếu học thuật: Kết hợp Stanford CS224N (Lý thuyết Alignment, Toán học Tối ưu hóa & Reasoning)

và Stanford CS336 (Kiến trúc Hệ thống & Giải phẫu Bộ nhớ từ Nguyên lý Gốc)

Tài liệu Giảng dạy Học thuật Chuyên sâu (Thời lượng: 180 Phút)

Thiết kế dành cho người học đã nắm vững Pre-training cơ bản, bắt đầu từ con số 0 với Post-Training

Phiên bản Chuẩn hóa Toán học & Trích dẫn Học thuật – Năm 2026

Tóm tắt nội dung

Bộ giáo trình này cung cấp một hệ thống lý thuyết và dẫn xuất toán học toàn diện về **Post-Training cho Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs)**. Tài liệu được thiết kế đặc biệt cho đối tượng người học mới chỉ được trang bị kiến thức cơ bản về giai đoạn Tiền huấn luyện (*Pre-training*: kiến trúc Transformer Decoder-only, cơ chế Tự chú ý Nhân quả *Causal Self-Attention*, bộ nhớ đệm *KV-Cache*, và hàm mất mát *Cross-Entropy* dự đoán từ tiếp theo), nhưng **chưa từng có kiến thức nền tảng về Post-Training hay Học tăng cường (Reinforcement Learning – RL)**. Toàn bộ nội dung loại bỏ sự phụ thuộc vào mã nguồn lập trình, tập trung giải phẫu bản chất vật lý và chứng minh từng bước mọi công thức toán học từ nguyên lý gốc (*First Principles*), đồng thời đối chiếu nghiêm ngặt với hơn 55 công trình nghiên cứu nền tảng từ Stanford, OpenAI, Anthropic, Meta, Google DeepMind, AI2 và DeepSeek.

Nội dung tài liệu bao gồm 6 chương chuyên sâu:

1. **Chương 1 – Tinh chỉnh có Giám sát (Supervised Fine-Tuning – SFT):** Phân tích khoảng cách mục tiêu (*Objective Mismatch*) qua phân kỳ KL xuôi, cấu trúc hội thoại và toán học khởi tạo ma trận nhúng cho Special Tokens, mặt nạ mất mát (*Prompt Masking*), đóng gói chuỗi (*Sequence Packing*) với mặt nạ chú ý đường chéo khối (*Block-Diagonal Causal Attention*), đặt lại vị trí quay RoPE, và quy luật dữ liệu từ LIMA đến Llama 3 / Tülu 3.

2. **Chương 2 – Nhập môn RL cho LLM, RLHF & Giải thuật PPO:** Giải thích ba giới hạn toán học của SFT (Ảo giác do bắt chước hành vi, Độ lệch phơi nhiễm bậc hai $\mathcal{O}(T^2)$ và Sự bất đối xứng Sinh–Kiểm chứng); xây dựng cầu nối nhập môn Học tăng cường từ hàm Cross-Entropy sang Đạo hàm Chính sách (*Policy Gradient*), chứng minh Bổ đề Đường cơ sở Không chệch (*Unbiased Baseline*); dẫn xuất mô hình Bradley–Terry và Plackett–Luce cho Reward Model, RLAIF / Constitutional AI; phân tích Định luật Goodhart và quy luật quá tối ưu hóa của Gao et al. (2023); chứng minh công thức GAE (*Generalized Advantage Estimation*), hàm mục tiêu cắt ngưỡng PPO-Clip, và giải phẫu bài toán hệ thống 4 mô hình trên VRAM.

3. **Chương 3 – Giải phẫu Toán học Direct Preference Optimization (DPO) & Các Biến thể:** Chứng minh đầy đủ nghiệm đóng Gibbs/Boltzmann của bài toán RL có ràng buộc KL, định lý về lớp tương đương của hàm phần thưởng, phép đổi biến triệt tiêu hàm phân hoạch $Z(x)$, phân tích động lực học của đạo hàm DPO; chứng minh toán học cho 4 cạm bẫy của DPO gốc và các biến thể khắc phục gồm **SimPO**, **ORPO**, **IPO**, **KTO**, và **Iterative RPO / DPO+NLL** chống hiện tượng dịch chuyển xác suất (*Likelihood Displacement*).

4. **Chương 4 – Kỷ nguyên Reasoning RL (RLVR, GRPO, DeepSeek-R1 & Test-Time Compute):** Phân tích sự dịch chuyển từ RLHF sang RLVR (*Reinforcement Learning with Verifiable Rewards*), cuộc tranh luận ORM vs. PRM, cơ chế tự phát sinh chuỗi suy luận dài (*Emergent Long CoT*) trong DeepSeek-R1-Zero và quy trình 4 giai đoạn của DeepSeek-R1; dẫn xuất toán học của **GRPO** cho cả giám sát kết quả và giám sát quá trình; chứng minh đầy đủ 3 bộ ước lượng KL của Schulman (k_1, k_2, k_3); phân tích thiên kiến độ dài và độ khó câu hỏi trong **Dr. GRPO** và **DAPO**; cùng toán học của mở rộng tính toán lúc suy luận (*Test-Time Compute Scaling*) và ước lượng không chệch Pass@k.

5. **Chương 5 – Chắt lọc Tri thức Mô hình (Model Distillation):** Phân tích toán học của mô hình Thầy–Trò trong Post-training qua 3 hình thái: Chắt lọc phân phối dự đoán (*Logit Distillation* và sự đối đầu giữa *Forward KL Mean-Seeking* vs. *Reverse KL Mode-Seeking* trong MiniLLM và On-Policy GKD), Chắt lọc cấp độ chuỗi (*Sequence-Level Distillation*), và Chắt lọc quá trình suy luận (*Rationale / Step-by-Step Distillation*).

6. **Chương 6 – Kiến trúc Bộ nhớ Hệ thống, Đánh giá Mô hình & Chẩn đoán Lỗi:** Giải phẫu từng byte bộ nhớ phần cứng (Model States, AdamW FP32 Master Weights, KV-Cache, Activations), phân tích Cường độ Số học (*Arithmetic Intensity*), toán học của ZeRO/FSDP, Gradient Checkpointing và LoRA/QLoRA; phân tích lý thuyết thông tin của Thuế căn chỉnh (*Alignment Tax*), mô hình thống kê Elo/Bradley–Terry trên Chatbot Arena, hồi quy khử thiên kiến độ dài trong **LC-AlpacaEval**, và cẩm nang chẩn đoán các bất thường toán học khi huấn luyện.

Mục lục

Khung Chương Trình & Lịch Trình Giảng Dạy (180 Phút) **5**

1 Từ Pre-training đến Tinh chỉnh có Giám sát (Supervised Fine-Tuning – SFT) **8**

1.1 Khoảng cách Mục tiêu (Objective Mismatch): Tại sao Mô hình Pre-trained chưa phải là Trợ lý AI? . . . 8

1.1.1 Hệ quả Vật lý của Phân kỳ KL Xuôi (Mean-Seeking / Mass-Covering) . . 9

1.2 Cấu trúc Dữ liệu Hội thoại, Chat Templates & Toán học Khởi tạo Nhúng 10

1.2.1 Tính Nguyên tử của Special Tokens và Cơ chế Chống Giả mạo Lệnh (Prompt Injection) . . . 11

1.2.2 Toán học của Khởi tạo Ma trận Nhúng khi Thêm Special Tokens [24] . . 11

1.3 Cơ chế Mặt nạ Mất mát (Prompt Masking / Loss Masking) . . . 12

1.3.1 Hai Hệ quả Sống còn của Mặt nạ Mất mát trong SFT . . . 13

1.4 Đóng gói Chuỗi (Sequence Packing), Mặt nạ Đường chéo Khối & Đặt lại Vị trí RoPE . . . 14

1.4.1 Điều kiện 1: Mặt nạ Nhân quả Đường chéo Khối (Block-Diagonal Causal Attention Mask) . . . 14

1.4.2 Điều kiện 2: Đặt lại Chỉ số Vị trí cho Mã hóa Vị trí Quay (RoPE Position Reset) . . . 15

1.5 Quy luật Dữ liệu trong SFT: Giả thuyết LIMA vs. Quy mô Công nghiệp Hiện đại 16

1.5.1 Giả thuyết Căn chỉnh Bề mặt (Superficial Alignment Hypothesis – LIMA [73]) . . . 16

1.5.2 Sự Mở rộng Quy mô SFT trong Thực tế Công nghiệp Hiện đại [17, 35] . . 16

2 Nhập môn Học Tăng Cường cho LLM, RLHF & Giải thuật PPO **17**

2.1 Ba Giới hạn Toán học và Nhận thức Cốt lõi của SFT . . . 17

2.1.1 1. Học Bắt chước Hành vi Kích hoạt Hiện tượng Ảo giác (Behavior Cloning Triggers Hallucinations) . . . 17

2.1.2 2. Độ lệch Phơi nhiễm (Exposure Bias) và Tích lũy Sai số Bậc hai $\mathcal{O}(T^2)$ 18

2.1.3 3. Sự Bất đối xứng Sinh–Kiểm chứng & Phạt Cứng nhắc ở Mức Token . . 19
2.2 Nhập môn Học Tăng Cường (RL) từ Con số 0 dưới Góc nhìn LLM 19
2.2.1 Bảng Ánh xạ Khái niệm Toán học giữa RL và LLM 20
2.2.2 Dẫn xuất Toán học của Định lý Đạo hàm Chính sách (Policy Gradient Theorem) 20
2.2.3 Bổ đề Đường Cơ sở Không chệch (Unbiased Baseline Lemma) 22
2.3 Mô hình hóa Sở thích Con người (Bradley–Terry) & Huấn luyện Reward Model . 22
2.3.1 Tại sao dùng So sánh Cặp (Pairwise Comparison)? 22
2.3.2 Mô hình Xác suất Bradley–Terry & Hàm Mất mát của Reward Model . . 23
2.3.3 Mở rộng cho So sánh Đa Ứng viên: Mô hình Plackett–Luce [40, 47] 23
2.3.4 Kiến trúc Transformer của Reward Model: Scalar Value Head tại Token Cuối 24
2.3.5 RLAIF & Constitutional AI [6, 36] 24
2.4 Tối ưu hóa Chính sách với Ràng buộc KL, Định luật Goodhart & PPO 24
2.4.1 Định luật Goodhart & Quy luật Quá Tối ưu hóa Phần thưởng (Reward Model Overoptimization Gao et al. 20) 25
2.4.2 Hàm Mục tiêu RLHF với Ràng buộc Phân kỳ KL 25
2.5 Giải thuật PPO & Generalized Advantage Estimation (GAE) 26
2.5.1 Bài toán Quy kết Công trạng & Mạng Phê bình (Critic V_ψ) 26
2.5.2 Ước lượng Lợi thế Tổng quát (GAE – Schulman et al., 2016) 26
2.5.3 Hàm Mục tiêu Cắt ngưỡng của PPO (PPO-Clip Schulman et al. 55) . . . 27
2.6 Kiến trúc Hệ thống 4 Mô hình trong PPO & Xung đột Phần cứng 28
2.6.1 Bài toán 4 Mô hình trên VRAM 28
2.6.2 Sự Xung đột Bản chất Phần cứng giữa Hai Pha trong Vòng lặp PPO . . 28

3 Giải phẫu Toán học Direct Preference Optimization (DPO) & Các Biến thể 30
3.1 Dẫn xuất Toán học Toàn vẹn của DPO (Rafailov et al., 2023) 30
3.1.1 Bước 1: Tìm Nghiệm Đóng (Closed-form Solution) của Bài toán RL có Ràng buộc KL 30
3.1.2 Bước 2: Phép Đổi biến Đột phá – Biểu diễn Ngược Hàm Phần thưởng theo Chính sách Tối ưu 32
3.1.3 Bước 3: Thế vào Mô hình Bradley–Terry & Sự Triệt tiêu Kỳ diệu của Hàm Phân hoạch $Z(x)$ 32
3.2 Giải phẫu Động lực học Gradient của DPO 33
3.2.1 Ý nghĩa Cơ học của Ba Thành phần trong Gradient DPO 34

3.3 Bốn Cạm bẫy Lý thuyết của DPO Gốc & Các Biến thể Khắc phục 34
3.3.1 Cạm bẫy 1: Thiên kiến Độ dài (Length / Verbosity Bias) & Giải pháp SimPO [41, 46] 34
3.3.2 Cạm bẫy 2: Hiện tượng Dịch chuyển Xác suất (Likelihood Displacement) & RPO [45, 51] 35
3.3.3 Cạm bẫy 3: Phân rã Đơn nhất không cần Reference Model (ORPO Hong et al. 27) 36
3.3.4 Cạm bẫy 4: Suy biến khi Sở thích Tất định (IPO) & Tối ưu theo Lý thuyết Triển vọng (KTO) 36
3.3.5 Offline DPO vs. Online / Iterative DPO [69] 37

4 Kỷ nguyên Reasoning RL: RLVR, Giải thuật GRPO, DeepSeek-R1 & Test-Time Compute 38

4.1 Sự Dịch chuyển Mô thức: Từ RLHF sang RLVR (Reinforcement Learning with Verifiable Rewards) 38
4.2 Cuộc Tranh Luận ORM vs. PRM & Hiện tượng Tự phát trong DeepSeek-R1-Zero 39
4.2.1 1. PRM (Process Reward Model – Lightman et al., OpenAI, 2023 Lightman et al. 37) 39
4.2.2 2. ORM / Rule-based Outcome Verifier & Thí nghiệm Lịch sử của DeepSeek-R1-Zero [15] 40
4.2.3 Sự Trỗi dậy Tự nhiên của Chuỗi Suy luận Dài & Khoảnh khắc "Aha" . . 40
4.2.4 Từ R1-Zero đến Quy trình 4 Giai đoạn Chuẩn mực của DeepSeek-R1 . . 40
4.3 Giải thuật GRPO (Group Relative Policy Optimization – Shao et al., 2024) . . . 41
4.3.1 Tại sao PPO Thất bại với Reasoning Chuỗi Dài? 41
4.3.2 Ý tưởng Cốt lõi của GRPO: Xóa sổ Hoàn toàn Mạng Critic! 41
4.3.3 Bốn Bước Toán học của Giải thuật GRPO 41
4.3.4 Mở rộng GRPO cho Giám sát Quá trình (Process Supervision) 42
4.4 Chứng minh Ba Bộ Ước lượng KL của Schulman (k_1, k_2, k_3) 42
4.5 Hai Thiên kiến Toán học trong GRPO Gốc, Dr. GRPO & DAPO 44
4.5.1 1. Thiên kiến Độ dài Câu trả lời (Response-level Length Bias) do Hệ số $\frac{1}{|o_i|}$ 44
4.5.2 2. Thiên kiến Độ khó Câu hỏi (Question-level Difficulty Bias) do Phép chia std(r) 44
4.5.3 Giải pháp của Dr. GRPO [38] & DAPO [70] 44
4.6 Quy luật Mở rộng Tính toán lúc Suy luận (Test-Time Compute Scaling) 45
4.6.1 Mở rộng Song song vs. Mở rộng Tuần tự 45
4.6.2 Công thức Ước lượng Không chệch Pass@k của Chen et al. (2021) 45

5 Chắt lọc Tri thức Mô hình (Model Distillation) trong Post-Training **46**

5.1 Mô hình Thầy–Trò trong Post-Training: Động lực & Khái niệm 46

5.2 Hình thái 1: Chắt lọc Phân phối Xác suất (Logit Distillation) & Nhiệt độ τ . . . 47

5.2.1 Hàm Softmax với Tham số Nhiệt độ $\tau > 0$ 47

5.3 Cuộc Đối đầu Toán học: Forward KL (Mean-Seeking) vs. Reverse KL (Mode-Seeking) 48

5.3.1 1. Phân kỳ KL Xuôi (Forward KL / Zero-Avoiding / Mean-Seeking) . . . 48

5.3.2 2. Phân kỳ KL Ngược (Reverse KL / Zero-Forcing / Mode-Seeking) . . . 48

5.3.3 3. Chắt lọc Trực tuyến trên Chính Sai lầm của Học sinh (On-Policy Distillation – MiniLLM & GKD) 49

5.4 Hình thái 2: Chắt lọc Cấp độ Chuỗi (Sequence-Level Distillation – Kim & Rush, 2016) 50

5.5 Hình thái 3: Chắt lọc Quá trình Suy luận (Rationale / Step-by-Step Distillation) 50

6 Kiến trúc Bộ nhớ Hệ thống, Đánh giá Mô hình & Chẩn đoán Lỗi **52**

6.1 Giải phẫu Bộ nhớ Phần cứng (VRAM Anatomy) từ Nguyên lý Gốc 52

6.1.1 1. Bộ nhớ Trạng thái Mô hình Tĩnh (Model States) 52

6.1.2 2. Bộ nhớ Động: KV-Cache & Trạng thái Kích hoạt Trung gian (Activations) 53

6.1.3 Cường độ Số học (Arithmetic Intensity) & Sự Nghẽn Cổ chai Phần cứng . 54

6.2 Bốn Trụ cột Tối ưu Hóa Hệ thống Post-Training Hiện đại 54

6.2.1 1. Tính trước Log-Probability Tham chiếu (Precomputing Reference Log-probs) 54

6.2.2 2. Tái tính toán Kích hoạt (Activation / Gradient Checkpointing Chen et al. 11) 54

6.2.3 3. Phân mảnh Trạng thái Mô hình (ZeRO-3 & FSDP Rajbhandari et al. 49, Zhao et al. 71) 55

6.2.4 4. Thích nghi Hạng Thấp (LoRA & QLoRA Dettmers et al. 16, Hu et al. 29) 55

6.3 Thuế Căn chỉnh (Alignment Tax) & Hệ sinh thái Đánh giá Mô hình 55

6.3.1 Tại sao không thể dùng Perplexity? Khái niệm Thuế Căn chỉnh (Alignment Tax) 55

6.3.2 Đánh giá Mở qua Giám khảo AI (LLM-as-a-Judge) & Ba Thiên kiến Nghiêm trọng 56

6.3.3 Các Bộ Tiêu chuẩn Đánh giá Kiểm chứng Khách quan (Verifiable Benchmarks) 56

6.4 Bảng Chẩn đoán Lỗi Thực chiến cho Nghiên cứu Sinh Post-Training 57

Khung Chương Trình & Lịch Trình Giảng Dạy (180 Phút)

1. Phân bổ Thời lượng Giảng dạy trên Lớp

2. Bảng Ký hiệu Toán học Thống nhất trong Toàn bộ Giáo trình

Ký hiệu	Ý nghĩa Toán học Chuẩn hóa
$V, \|V\|$	Bảng từ vựng (*Vocabulary*) của mô hình và kích thước bảng từ vựng (số lượng token).
d	Số chiều không gian ẩn (*Hidden dimension*) của kiến trúc Transformer.
$\mathbf{E}_{\text{in}}, \mathbf{E}_{\text{out}} \in \mathbb{R}^{\|V\| \times d}$	Ma trận nhúng từ vựng đầu vào (*Input Embedding*) và ma trận chiếu từ vựng đầu ra (*LM Head*).
$x = (x_1, \ldots, x_M)$	Chuỗi token đầu vào (*Prompt / Context / Question*), đôi khi ký hiệu là q trong bài toán suy luận.
$y = (y_1, \ldots, y_T)$	Chuỗi token phản hồi (*Response / Completion / Output*), đôi khi ký hiệu là o trong GRPO, với độ dài $\|y\| = T$.
$y_{<t} = (y_1, \ldots, y_{t-1})$	Tiền tố các token đã được sinh ra trước bước thời gian t (với quy ước $y_{<1} = \emptyset$).
$\pi_\theta(y_t \mid x, y_{<t})$	Phân phối xác suất dự đoán token thứ t của mô hình chính sách (*Policy / Actor*) có tham số θ.
$\pi_{\text{SFT}}, \pi_{\text{ref}}$	Mô hình sau tinh chỉnh có giám sát và mô hình tham chiếu đóng băng (*Frozen Reference Policy*).
$m_t \in \{0, 1\}$	Mặt nạ mất mát nhị phân (*Loss Mask*) tại vị trí t: $m_t = 1$ nếu thuộc phần trả lời của Assistant, $m_t = 0$ nếu thuộc Prompt.
$y_w \succ y_l \mid x$	Phản hồi được chọn (*winner / chosen* y_w) được đánh giá cao hơn phản hồi bị loại (*loser / rejected* y_l) cho câu hỏi x.
$r_\phi(x, y) \in \mathbb{R}$	Điểm thưởng vô hướng xuất ra từ Mô hình Phần thưởng (*Reward Model*) có bộ tham số ϕ.
$V_\psi(x, y_{<t}) \in \mathbb{R}$	Giá trị kỳ vọng tương lai ước lượng tại trạng thái $(x, y_{<t})$ bởi Mạng Phê bình (*Critic / Value Model*) có tham số ψ.
$A_t \in \mathbb{R}$	Giá trị Hàm Lợi thế (*Advantage Function*) tại bước thời gian t.

Ký hiệu	**Ý nghĩa Toán học Chuẩn hóa**
$\delta_t^V \in \mathbb{R}$	Sai số dự đoán theo thời gian (*Temporal-Difference Residual*): $\delta_t^V = \tilde{r}_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$.
$\mathbb{D}_{\mathrm{KL}}(P \parallel Q)$	Phân kỳ Kullback–Leibler từ phân phối Q sang phân phối P: $\mathbb{D}_{\mathrm{KL}}(P \parallel Q) = \mathbb{E}_{z\sim P}\left[\log(P(z)/Q(z))\right] \geq 0$.
$\beta > 0$	Hệ số điều phạt độ lệch phân kỳ KL (*KL-Penalty Coefficient*) giữ π_θ không đi quá xa π_{ref}.
$Z(x)$	Hàm phân hoạch (*Partition Function*): $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y)\right)$.
$\sigma(z)$	Hàm Logistic Sigmoid chuẩn: $\sigma(z) = \frac{1}{1+e^{-z}}$, thỏa mãn $1-\sigma(z) = \sigma(-z)$ và $\frac{d}{dz}\log\sigma(z) = \sigma(-z)$.
$\rho_t(\theta)$	Tỷ số lấy mẫu quan trọng (*Importance Sampling Ratio*): $\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\mathrm{old}}}(y_t \mid x, y_{<t})}$.

Thời lượng	Học phần	Trọng tâm Toán học, Lý thuyết & Hệ thống	Các Công trình Nghiên cứu Cốt lõi
00:00 – 00:30 *(30 phút)*	**Chương 1: Từ Pre-training đến SFT**	Phân kỳ KL xuôi & Objective Mismatch, Toán học khởi tạo Embedding cho Special Tokens, Mặt nạ mất mát (Prompt Masking), Đóng gói chuỗi (Sequence Packing), Mặt nạ đường chéo khối & RoPE Reset, LIMA vs. Scaling SFT	Brown et al. [9], Dubey et al. [17], Hewitt [24], Lambert et al. [35], Ouyang et al. [43], Su et al. [60], Zhou et al. [73]
00:30 – 01:10 *(40 phút)*	**Chương 2: Nhập môn RL, RLHF & PPO**	Ba giới hạn của SFT (Hallucination, Exposure Bias $\mathcal{O}(T^2)$), Cầu nối từ Cross-Entropy sang Policy Gradient & Baseline không chệch, Bradley–Terry, Plackett–Luce, RLAIF, Định luật Goodhart, GAE, PPO-Clip & Bài toán 4 mô hình	Bai et al. [6], Bradley and Terry [7], Christiano et al. [13], Gao et al. [20], Kang et al. [33], Schulman et al. [54, 55], Stiennon et al. [59], Williams [68]
01:10 – 01:45 *(35 phút)*	**Chương 3: Giải phẫu Toán học DPO & Biến thể**	Nghiệm đóng Gibbs/Boltzmann, Lớp tương đương phần thưởng, Triệt tiêu hàm phân hoạch $Z(x)$, Động lực học đạo hàm DPO, Bốn cạm bẫy lý thuyết (Length Bias, Likelihood Displacement) & SimPO, ORPO, IPO, KTO, RPO	Azar et al. [4], Ethayarajh et al. [19], Hong et al. [27], Meng et al. [41], Pang et al. [45], Park et al. [46], Rafailov et al. [48], Razin et al. [51]
01:45 – 02:20 *(35 phút)*	**Chương 4: Reasoning RL, RLVR & GRPO**	RLVR vs. RLHF, ORM vs. PRM, Sự trỗi dậy tự nhiên của CoT trong DeepSeek-R1-Zero & Quy trình 4 bước của R1, Toán học GRPO (Outcome & Process), Chứng minh 3 bộ ước lượng KL của Schulman, Dr. GRPO, DAPO & Test-Time Compute	Chen et al. [10], DeepSeek-AI et al. [15], Lightman et al. [37], Liu et al. [38], Schulman [53], Shao et al. [56], Snell et al. [58], Yu et al. [70]
02:20 – 02:40 *(20 phút)*	**Chương 5: Chắt lọc Tri thức (Model Distillation)**	Mô hình Thầy–Trò, Chắt lọc phân phối Logits với nhiệt độ τ, Chứng minh Forward KL (Mean-Seeking) vs. Reverse KL (Mode-Seeking) trong MiniLLM & On-Policy GKD, Chắt lọc chuỗi (SeqKD) & Chắt lọc suy luận (Rationale KD)	Agarwal et al. [1], DeepSeek-AI et al. [15], Gu et al. [22], Hinton et al. [25], Hsieh et al. [28], Kim and Rush [34]
02:40 – 03:00 *(20 phút)*	**Chương 6: Hệ thống, Đánh giá & Chẩn đoán**	Giải phẫu từng byte VRAM (Model States, KV-Cache, Activations), Cường độ số học, Toán học của ZeRO/FSDP, Checkpointing & LoRA, Alignment Tax, Hồi quy khử thiên kiến LC-AlpacaEval & Bảng chẩn đoán lỗi toán học	Chiang et al. [12], Dettmers et al. [16], Dubois et al. [18], Hu et al. [29], Rajbhandari et al. [49], Sheng et al. [57], Zheng et al. [72]

Bảng 1: Lịch trình phân bổ 180 phút giảng dạy lý thuyết và toán học Post-Training.

Chương 1

Từ Pre-training đến Tinh chỉnh có Giám sát (Supervised Fine-Tuning – SFT)

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 1.1. Chương này xuất phát trực tiếp từ kiến thức nền tảng duy nhất mà người học đã có: **giai đoạn Tiền huấn luyện (Pre-training)** với kiến trúc Transformer Decoder-only [64], cơ chế Tự chú ý Nhân quả (*Causal Self-Attention*) và hàm mất mát Cross-Entropy dự đoán token tiếp theo. Chúng ta sẽ chứng minh bằng toán học tại sao một mô hình vừa bước ra từ Pre-training chưa thể làm trợ lý AI, và làm thế nào để điều chỉnh hàm mất mát, ma trận nhúng cùng mặt nạ chú ý trong giai đoạn **Tinh chỉnh có Giám sát (Supervised Fine-Tuning – SFT)** nhằm định hình hành vi trợ lý.

1.1 Khoảng cách Mục tiêu (Objective Mismatch): Tại sao Mô hình Pre-trained chưa phải là Trợ lý AI?

Trong giai đoạn Tiền huấn luyện (*Pre-training*), một mô hình ngôn ngữ lớn với bộ tham số θ được huấn luyện trên kho văn bản thô thu thập từ Internet $\mathcal{D}_{\text{Pretrain}}$ (bao gồm hàng chục nghìn tỷ token từ sách báo, bài viết khoa học, diễn đàn, mã nguồn, đề thi...). Với mỗi chuỗi văn bản độ dài T là $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ trong đó mỗi token $x_t \in V$ thuộc bảng từ vựng V, theo quy tắc nhân xác suất (*Chain Rule of Probability*), xác suất đồng thời của cả chuỗi được phân rã tự hồi quy (*Autoregressive Factorization*) thành:

$$P_\theta(\mathbf{x}) = P_\theta(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, x_2, \ldots, x_{t-1}) = \prod_{t=1}^{T} P_\theta(x_t \mid x_{<t}). \tag{1.1}$$

Hàm mục tiêu của Pre-training là cực tiểu hóa hàm mất mát **Negative Log-Likelihood**

(Cross-Entropy) trung bình trên phân phối dữ liệu thực nghiệm của Internet $P_{\text{Internet}}(\mathbf{x})$:

$$\mathcal{L}_{\text{Pretrain}}(\theta) = \mathbb{E}_{\mathbf{x}\sim P_{\text{Internet}}}\left[-\sum_{t=1}^{T}\log P_\theta(x_t \mid x_{<t})\right] = -\,\mathbb{E}_{\mathbf{x}\sim P_{\text{Internet}}}\left[\log P_\theta(\mathbf{x})\right]. \tag{1.2}$$

Mệnh đề 1.1 (Sự tương đương giữa Pre-training Cross-Entropy và Phân kỳ KL Xuôi)**.** *Việc cực tiểu hóa hàm mất mát Pre-training $\mathcal{L}_{Pretrain}(\theta)$ trong Phương trình (1.2) theo tham số θ hoàn toàn tương đương về mặt toán học với việc cực tiểu hóa* ***Phân kỳ Kullback–Leibler xuôi (Forward KL Divergence)*** *từ phân phối mô hình P_θ sang phân phối văn bản Internet thực tế $P_{Internet}$:*

$$\arg\min_\theta \mathcal{L}_{Pretrain}(\theta) = \arg\min_\theta \mathbb{D}_{\text{KL}}\left(P_{Internet}(\mathbf{x}) \parallel P_\theta(\mathbf{x})\right). \tag{1.3}$$

Chứng minh. Theo định nghĩa của Phân kỳ Kullback–Leibler giữa hai phân phối xác suất rời rạc $P_{\text{Internet}}(\mathbf{x})$ và $P_\theta(\mathbf{x})$ trên không gian chuỗi V^T:

$$\begin{aligned}
\mathbb{D}_{\text{KL}}\left(P_{\text{Internet}}(\mathbf{x}) \parallel P_\theta(\mathbf{x})\right) &= \sum_{\mathbf{x}\in V^T} P_{\text{Internet}}(\mathbf{x}) \log \frac{P_{\text{Internet}}(\mathbf{x})}{P_\theta(\mathbf{x})} \\
&= \underbrace{\sum_{\mathbf{x}\in V^T} P_{\text{Internet}}(\mathbf{x}) \log P_{\text{Internet}}(\mathbf{x})}_{-\mathcal{H}(P_{\text{Internet}})\text{ (Hằng số không phụ thuộc }\theta)} - \sum_{\mathbf{x}\in V^T} P_{\text{Internet}}(\mathbf{x}) \log P_\theta(\mathbf{x}) \\
&= -\mathcal{H}(P_{\text{Internet}}) + \mathcal{L}_{\text{Pretrain}}(\theta),
\end{aligned} \tag{1.4}$$

trong đó $\mathcal{H}(P_{\text{Internet}})$ là Entropy Shannon của dữ liệu Internet. Vì $\mathcal{H}(P_{\text{Internet}})$ không phụ thuộc vào θ, việc lấy đạo hàm theo θ cho $\nabla_\theta \mathbb{D}_{\text{KL}}(P_{\text{Internet}} \parallel P_\theta) = \nabla_\theta \mathcal{L}_{\text{Pretrain}}(\theta)$. Do $\mathbb{D}_{\text{KL}} \geq 0$, nghiệm tối ưu toàn cục đạt được khi và chỉ khi $P_\theta(\mathbf{x}) = P_{\text{Internet}}(\mathbf{x})$ với mọi chuỗi $\mathbf{x}$. □

1.1.1 Hệ quả Vật lý của Phân kỳ KL Xuôi (Mean-Seeking / Mass-Covering)

Từ Mệnh đề 1.1, hãy quan sát biểu thức $\sum_{\mathbf{x}} P_{\text{Internet}}(\mathbf{x}) \log \frac{P_{\text{Internet}}(\mathbf{x})}{P_\theta(\mathbf{x})}$:

- Bất cứ chuỗi văn bản $\mathbf{x}$ nào có xuất hiện trên Internet ($P_{\text{Internet}}(\mathbf{x}) > 0$), nếu mô hình gán cho nó xác suất $P_\theta(\mathbf{x}) \to 0$, thì tỷ số $\frac{P_{\text{Internet}}(\mathbf{x})}{P_\theta(\mathbf{x})} \to +\infty$, đẩy hàm mất mát $\mathbb{D}_{\text{KL}} \to +\infty$.
- Tính chất này buộc mô hình Pre-training phải **trải rộng khối lượng xác suất để bao phủ mọi kiểu văn bản tồn tại trên Internet** (*Mass-Covering / Mean-Seeking property*).

Hệ quả trực tiếp là mô hình nền tảng sau Pre-training (**Base Model**, ví dụ GPT-3 gốc của Brown et al. 9) thực chất là một **Bộ hoàn thành tài liệu xác suất (Probabilistic Document Completer)** chứ không phải là một trợ lý giải quyết yêu cầu của con người.

Ví dụ Minh họa 1.1 (Hiện tượng Hoàn thành Đề thi của Base Model)**.** Khi người dùng nhập đầu vào $\mathbf{x}_{\text{prompt}}$: *"Câu 1: Giải thích ngắn gọn định luật II Newton cho học sinh cấp 2."* Thay vì viết lời giải thích, Base Model tính toán phân phối có điều kiện $P_{\text{Internet}}(x_t \mid \mathbf{x}_{\text{prompt}})$. Bởi vì trên mạng Internet, một câu bắt đầu bằng *"Câu 1: Giải thích..."* xuất hiện với tần suất cao

nhất trong các tờ đề cương ôn tập hoặc đề thi Vật lý chưa có lời giải, chuỗi tiếp theo có xác suất lớn nhất dưới phân phối P_{Internet} sẽ là:

> *"Câu 2: Phát biểu định luật III Newton và nêu ví dụ thực tế. Câu 3: Một vật có khối lượng $m = 5$ kg chịu tác dụng của lực..."*

Trong công trình nền tảng **InstructGPT**, Ouyang et al. [43] định nghĩa hiện tượng này là **Khoảng cách Mục tiêu (Objective Mismatch)** giữa:

1. **Mục tiêu toán học của Pre-training:** Tối đa hóa hợp lý dự đoán token tiếp theo trên phân phối văn bản hỗn tạp của Internet.
2. **Mục tiêu thực tế của người dùng:** Mong muốn mô hình hành xử như một trợ lý tuân thủ chỉ thị theo ba tiêu chuẩn **HHH** của Askell et al. [3]: **Hữu ích (Helpful)**, **Trung thực (Honest)** và **Vô hại (Harmless)**.

Do đó, bản chất cốt lõi của toàn bộ quá trình **Post-Training** không phải là nạp thêm tri thức sự kiện mới vào mô hình (vì tri thức thế giới và cấu trúc suy luận tiềm ẩn đã được hình thành trong trọng số từ Pre-training), mà là **Căn chỉnh Hành vi (Behavioral Alignment)** và **Khơi gợi Năng lực (Capability Elicitation)**: thu hẹp phân phối xác suất rộng lớn của Base Model để tập trung vào vùng không gian của các câu trả lời chất lượng cao.

1.2 Cấu trúc Dữ liệu Hội thoại, Chat Templates & Toán học Khởi tạo Nhúng

Trong Pre-training, dữ liệu là một chuỗi văn bản phẳng, liên tục, chỉ được phân tách bởi token bắt đầu văn bản ⟨BOS⟩ và kết thúc văn bản ⟨EOS⟩. Để mô hình có thể tham gia vào một cuộc đối thoại đa lượt (*Multi-turn Conversation*), văn bản cần được phân định thành ba vai trò ngữ nghĩa riêng biệt:

- **System (Hệ thống):** Đưa ra quy tắc nền tảng, phạm vi nhiệm vụ và ràng buộc hành vi toàn cục.
- **User (Người dùng):** Đưa ra câu hỏi, yêu cầu hoặc bài toán tại từng lượt hội thoại.
- **Assistant (Trợ lý AI):** Phần văn bản do chính mô hình sinh ra để trả lời người dùng.

Vì mạng Transformer chỉ nhận đầu vào là một chuỗi một chiều các mã định danh token, ta cần thiết kế một giao thức tuần tự hóa gọi là **Khung định dạng hội thoại (Chat Template)** sử dụng các **Token Điều khiển Đặc biệt (Special Control Tokens)** như ⟨|im_start|⟩ và ⟨|im_end|⟩:

```
⟨|im_start|⟩system
Bạn là trợ lý AI hữu ích và chính xác.⟨|im_end|⟩
⟨|im_start|⟩user
```

`Thủ đô của Pháp là gì?`⟨|`im_end`|⟩
⟨|`im_start`|⟩`assistant`
`Thủ đô của Pháp là Paris.`⟨|`im_end`|⟩

1.2.1 Tính Nguyên tử của Special Tokens và Cơ chế Chống Giả mạo Lệnh (Prompt Injection)

Tại sao ta không ghép các ký tự có sẵn trong bảng từ vựng như '`<`', '`|`', '`i`', '`m`' mà phải mở rộng bảng từ vựng V để cấp cho cả cụm ⟨|im_start|⟩ một mã định danh nguyên tử (*Atomic Token ID*) duy nhất?

1. **Tính phân tách xác suất tuyệt đối:** Một Token ID mới hoàn toàn chưa từng xuất hiện trong các đoạn văn bản thông thường của Pre-training, giúp cơ chế Self-Attention nhận diện ranh giới chuyển vai trò với độ chắc chắn tuyệt đối.

2. **An toàn hệ thống trước tấn công giả mạo vai trò:** Khi người dùng nhập văn bản vào khung trò chuyện, bộ phân tách từ (*Tokenizer*) ở chế độ đầu vào người dùng sẽ không cho phép ánh xạ chuỗi ký tự '`<|im_start|>system`' thành Token ID điều khiển nguyên tử, mà băm nó thành các token ký tự rời rạc. Nhờ đó, người dùng không thể tự ý đóng giả thẻ điều khiển của 'system' hoặc 'assistant'.

1.2.2 Toán học của Khởi tạo Ma trận Nhúng khi Thêm Special Tokens [24]

Giả sử bảng từ vựng gốc của Base Model có $|V|$ token, tương ứng với ma trận nhúng đầu vào $\mathbf{E}_{\text{in}} \in \mathbb{R}^{|V|\times d}$ và ma trận chiếu đầu ra (*Language Modeling Head*) $\mathbf{E}_{\text{out}} \in \mathbb{R}^{|V|\times d}$, trong đó hàng thứ i là vector $\mathbf{e}_i \in \mathbb{R}^d$. Khi mở rộng bảng từ vựng thêm K token đặc biệt mới, kích thước ma trận tăng lên thành $(|V|+K)\times d$.

Định lý 1.2 (Bùng nổ Đạo hàm do Khởi tạo Phương sai Lớn và Giải pháp Khởi tạo Trung bình [24])**.** *Giả sử các vector nhúng đã qua Pre-training $\{\mathbf{e}_i\}_{i=1}^{|V|}$ có kỳ vọng thực nghiệm $\boldsymbol{\mu} = \frac{1}{|V|}\sum_{i=1}^{|V|}\mathbf{e}_i$ và hiệp phương sai thực nghiệm $\boldsymbol{\Sigma}$ với chuẩn trung bình nhỏ ($\|\mathbf{e}_i\|_2 \approx \sqrt{d}\,\sigma_{pre}$, thường $\sigma_{pre} \approx 0.02$).*

(a) Nếu vector của token mới $\mathbf{e}_{new}$ được khởi tạo ngẫu nhiên theo phân phối chuẩn $\mathcal{N}(\mathbf{0}, \sigma_{init}^2\mathbf{I}_d)$ với $\sigma_{init} \gg \sigma_{pre}$ (ví dụ $\sigma_{init} = 1$), thì với vector ẩn tầng cuối $\mathbf{h}_t \in \mathbb{R}^d$, phương sai của logit mới $z_{new} = \mathbf{h}_t^\top \mathbf{e}_{new}$ là $\mathrm{Var}(z_{new}) = \|\mathbf{h}_t\|_2^2\sigma_{init}^2 \gg \mathrm{Var}(z_i)$. Khi $z_{new} > 0$ lớn, xác suất dự đoán cho token mới $P_\theta(x_t = new \mid x_{<t}) \to 1$, làm hàm mất mát và chuẩn đạo hàm truyền ngược về $\mathbf{h}_t$ bùng nổ.

(b) Nếu khởi tạo $\mathbf{e}_{new}$ theo phân phối chuẩn có tâm tại vector trung bình cộng:

$$\mathbf{e}_{new} \sim \mathcal{N}\left(\boldsymbol{\mu},\ \frac{1}{|V|}\boldsymbol{\Sigma}\right) \quad \textit{hoặc xấp xỉ tất định } \mathbf{e}_{new} = \frac{1}{|V|}\sum_{i=1}^{|V|}\mathbf{e}_i, \tag{1.5}$$

thì tại bước khởi đầu của SFT, logit của token mới bằng đúng trung bình cộng logit của các token hiện có: $z_{new} = \mathbf{h}_t^\top \boldsymbol{\mu} = \frac{1}{|V|}\sum_{i=1}^{|V|} z_i$, đảm bảo xác suất tiên nghiệm của token mới có bậc độ lớn $\mathcal{O}\big(\frac{1}{|V|}\big)$ êm ái và bảo toàn phân phối gốc trên các token cũ.

Chứng minh. Xét tầng chiếu đầu ra tại bước thời gian t với vector trạng thái ẩn $\mathbf{h}_t \in \mathbb{R}^d$. Với mỗi token $i \in \{1, \dots, |V|\}$, giá trị logit tương ứng là $z_i = \mathbf{h}_t^\top \mathbf{e}_i$. Khi khởi tạo $\mathbf{e}_{\text{new}} = \boldsymbol{\mu} = \frac{1}{|V|}\sum_{i=1}^{|V|} \mathbf{e}_i$, do tính tuyến tính của tích vô hướng, giá trị logit dành cho token mới là:

$$z_{\text{new}} = \mathbf{h}_t^\top \mathbf{e}_{\text{new}} = \mathbf{h}_t^\top \left(\frac{1}{|V|} \sum_{i=1}^{|V|} \mathbf{e}_i \right) = \frac{1}{|V|} \sum_{i=1}^{|V|} (\mathbf{h}_t^\top \mathbf{e}_i) = \bar{z}. \tag{1.6}$$

Khi đó, với bất kỳ cặp token cũ $i, j \in \{1, \dots, |V|\}$ nào, tỷ số xác suất tương đối giữa chúng sau khi thêm K token mới vẫn được bảo toàn tuyệt đối:

$$\frac{P_\theta^{\text{expanded}}(x_t = i \mid x_{<t})}{P_\theta^{\text{expanded}}(x_t = j \mid x_{<t})} = \frac{\exp(z_i) / \big(\sum_{v=1}^{|V|} e^{z_v} + K e^{\bar{z}} \big)}{\exp(z_j) / \big(\sum_{v=1}^{|V|} e^{z_v} + K e^{\bar{z}} \big)} = \frac{\exp(z_i)}{\exp(z_j)} = \frac{P_\theta^{\text{orig}}(x_t = i \mid x_{<t})}{P_\theta^{\text{orig}}(x_t = j \mid x_{<t})}. \tag{1.7}$$

Ngược lại, nếu $\mathbf{e}_{\text{new}} \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{init}}^2 \mathbf{I}_d)$ với $\sigma_{\text{init}} \gg \sigma_{\text{pre}}$, do $\text{Var}(z_{\text{new}} \mid \mathbf{h}_t) = \|\mathbf{h}_t\|_2^2 \sigma_{\text{init}}^2$, với xác suất $1/2$ biến chuẩn z_{new} nhận giá trị dương cực lớn $z_{\text{new}} \gg \max_{i \in V} z_i$. Khi đó $\exp(z_{\text{new}})$ thống trị mẫu số của hàm Softmax, làm $P_\theta(\text{new} \mid x_{<t}) \approx 1$. Tại mọi vị trí t mà nhãn thực tế là một từ thông thường $x_t^* \in V$, đạo hàm của hàm mất mát Cross-Entropy $\ell_t = -\log P_\theta(x_t^* \mid x_{<t})$ theo vector ẩn $\mathbf{h}_t$ là:

$$\frac{\partial \ell_t}{\partial \mathbf{h}_t} = \sum_{v \in V \cup \{\text{new}\}} P_\theta(v \mid x_{<t}) \mathbf{e}_v - \mathbf{e}_{x_t^*} \approx \mathbf{e}_{\text{new}} - \mathbf{e}_{x_t^*}, \tag{1.8}$$

có chuẩn Euclid $\|\frac{\partial \ell_t}{\partial \mathbf{h}_t}\|_2 \approx \|\mathbf{e}_{\text{new}}\|_2 \approx \sqrt{d}\,\sigma_{\text{init}}$, lớn gấp $\frac{\sigma_{\text{init}}}{\sigma_{\text{pre}}}$ lần (hàng chục lần) so với đạo hàm chuẩn của mô hình, gây ra hiện tượng quên thảm khốc (*Catastrophic Forgetting*) trên toàn bộ các tầng Transformer phía dưới ngay trong những bước cập nhật đầu tiên. □

1.3 Cơ chế Mặt nạ Mất mát (Prompt Masking / Loss Masking)

Sau khi tuần tự hóa cuộc hội thoại thành một chuỗi token duy nhất $\mathbf{x} = (x_1, x_2, \dots, x_T)$, bước tiếp theo là định nghĩa hàm mục tiêu huấn luyện của SFT. Nếu áp dụng nguyên xi hàm mất mát Pre-training ở Phương trình (1.2), mô hình sẽ tính Cross-Entropy trên mọi vị trí $t \in \{1, \dots, T\}$. Tuy nhiên, trong một mẫu hội thoại $\mathbf{x} = (x_{\text{prompt}}, y_{\text{response}})$, phần đầu vào x_{prompt} (bao gồm chỉ dẫn của 'system' và câu hỏi của 'user') là dữ liệu cho trước bởi môi trường bên ngoài; nhiệm vụ của trợ lý AI chỉ là sinh ra phần phản hồi y_{response}.

Định nghĩa 1.1 (Hàm Mất mát SFT với Mặt nạ Nhị phân)**.** Với chuỗi hội thoại $\mathbf{x} = (x_1, x_2, \dots, x_T)$, ta định nghĩa dãy **Mặt nạ Mất mát (Loss Mask)** $\mathbf{m} = (m_1, m_2, \dots, m_T) \in \{0, 1\}^T$ theo quy tắc:

$$m_t = \begin{cases} 1 & \text{nếu token } x_t \text{ thuộc nội dung phản hồi của } \texttt{assistant} \text{ (kể cả token kết thúc } \langle|\text{im_end}|\rangle), \\ 0 & \text{nếu token } x_t \text{ thuộc } \texttt{system}, \texttt{user}, \text{ hoặc thẻ mở đầu } \langle|\text{im_start}|\rangle\texttt{assistant}. \end{cases} \tag{1.9}$$

Khi đó, hàm mất mát Tinh chỉnh có Giám sát $\mathcal{L}_{\mathrm{SFT}}(\theta)$ được định nghĩa là trung bình có trọng số mặt nạ của Negative Log-Likelihood:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{\sum_{t=1}^{T} m_t} \sum_{t=1}^{T} m_t \cdot \log \pi_\theta(x_t \mid x_1, \ldots, x_{t-1}). \tag{1.10}$$

Mệnh đề 1.3 (Sự chuyển đổi từ Xác suất Đồng thời sang Xác suất Có điều kiện qua Mặt nạ Mất mát)**.** *Xét một cuộc hội thoại đơn lượt gồm câu hỏi $x = (x_1, \ldots, x_M)$ và câu trả lời $y = (y_1, \ldots, y_N)$ được nối thành chuỗi $\mathbf{z} = (x_1, \ldots, x_M, y_1, \ldots, y_N)$ có tổng độ dài $T = M + N$. Với mặt nạ $m_t = 0$ cho $1 \le t \le M$ và $m_t = 1$ cho $M + 1 \le t \le M + N$, tổng mất mát trong Phương trình (1.10) chính là Negative Log-Likelihood có điều kiện $-\log \pi_\theta(y \mid x)$:*

$$-\sum_{t=1}^{M+N} m_t \log \pi_\theta(z_t \mid z_{<t}) = -\sum_{k=1}^{N} \log \pi_\theta(y_k \mid x, y_{<k}) = -\log \pi_\theta(y \mid x). \tag{1.11}$$

Chứng minh. Theo quy tắc Bayes cho chuỗi $\mathbf{z} = (x, y)$, logarit xác suất đồng thời là:

$$\log \pi_\theta(x, y) = \log \pi_\theta(x) + \log \pi_\theta(y \mid x) = \sum_{t=1}^{M} \log \pi_\theta(x_t \mid x_{<t}) + \sum_{k=1}^{N} \log \pi_\theta(y_k \mid x, y_{<k}). \tag{1.12}$$

Khi nhân từng hạng tử với m_t theo định nghĩa ($m_t = 0$ với $t \le M$ và $m_t = 1$ với $t > M$), tổng đầu tiên $\sum_{t=1}^{M} 0 \cdot \log \pi_\theta(x_t \mid x_{<t})$ bị triệt tiêu hoàn toàn, chỉ còn lại đúng $-\log \pi_\theta(y \mid x)$. □

1.3.1 Hai Hệ quả Sống còn của Mặt nạ Mất mát trong SFT

1. **Bất đối xứng giữa Lan truyền Xuôi (Attention) và Lan truyền Ngược (Loss):** Mặc dù $m_t = 0$ tại các vị trí thuộc câu hỏi x khiến các vị trí này không sinh ra đạo hàm trực tiếp ở hàm mất mát, nhưng trong pha lan truyền xuôi qua các tầng Self-Attention, các token thuộc câu trả lời y vẫn chú ý (*attend*) đầy đủ vào các vector Key và Value của câu hỏi x. Do đó, đạo hàm từ $\log \pi_\theta(y_k \mid x, y_{<k})$ vẫn truyền ngược qua các ma trận $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ tại các vị trí của x, giúp mô hình học cách trích xuất thông tin từ câu hỏi để phục vụ cho câu trả lời!

2. **Toán học của Điểm dừng Tự hồi quy (Stopping Criterion tại $\langle|\mathbf{im_end}|\rangle$):** Quá trình mô hình sinh câu trả lời $y = (y_1, \ldots, y_N)$ chỉ kết thúc tại bước N khi và chỉ khi mô hình chọn hành động xuất ra token kết thúc lượt $y_N = \langle|\mathrm{im_end}|\rangle$. Xác suất để mô hình sinh một câu trả lời có độ dài hữu hạn đúng bằng N là:

$$P(\text{Stop at } N \mid x, y_{<N}) = \pi_\theta\big(y_N = \langle|\mathrm{im_end}|\rangle \;\big|\; x, y_1, \ldots, y_{N-1}\big) \cdot \prod_{k=1}^{N-1} \Big(1 - \pi_\theta\big(\langle|\mathrm{im_end}|\rangle \mid x, y_{<k}\big)\Big). \tag{1.13}$$

   Nếu người thiết kế vô tình gán $m_N = 0$ tại vị trí token $\langle|\mathrm{im_end}|\rangle$ của Assistant, không có bất kỳ lực đạo hàm nào kéo $\pi_\theta(\langle|\mathrm{im_end}|\rangle \mid x, y_{<N})$ tăng lên khi câu trả lời đã hoàn chỉnh. Hệ quả là xác suất dừng luôn gần bằng 0, khiến mô hình sinh chữ bất tận cho đến khi tràn bộ nhớ ngữ cảnh. Ngược lại, tại vị trí $\langle|\mathrm{im_end}|\rangle$ ở cuối câu hỏi của User, ta

bắt buộc phải gán $m_t = 0$ vì người quyết định kết thúc câu hỏi là User chứ không phải Assistant.

1.4 Đóng gói Chuỗi (Sequence Packing), Mặt nạ Đường chéo Khối & Đặt lại Vị trí RoPE

Trong thực tế huấn luyện trên phần cứng xử lý ma trận song song, mỗi lô dữ liệu (*Batch*) là một tensor chữ nhật kích thước $B \times L_{\max}$ (với $L_{\max}$ là độ dài ngữ cảnh tối đa, ví dụ $8,192$ tokens). Trong tập dữ liệu SFT, độ dài L_k của từng cuộc hội thoại biến thiên rất lớn ($50 \le L_k \le 8,192$).

Nếu áp dụng phương pháp **Chèn đệm (Padding)** (điền các token rỗng $\langle \text{pad} \rangle$ vào cuối các câu ngắn cho đủ độ dài của câu dài nhất trong lô), tỷ lệ tính toán hữu ích (*Effective Token Ratio*) chỉ đạt:

$$\eta_{\text{pad}} = \frac{\sum_{b=1}^{B} L_b}{B \cdot \max_{1 \le b \le B} L_b} \ll 1 \quad \text{(thường chỉ đạt 15\% đến 30\%).} \tag{1.14}$$

Do bậc độ phức tạp tính toán của tầng Self-Attention tỷ lệ bậc hai $\mathcal{O}(B \cdot L_{\max}^2 \cdot d)$, việc nhân ma trận trên $70\% - 85\%$ các token $\langle \text{pad} \rangle$ gây lãng phí tài nguyên phần cứng nghiêm trọng.

Để đạt hiệu suất $\eta_{\text{pack}} \approx 100\%$, kỹ thuật **Đóng gói Chuỗi (Sequence Packing)** ghép nối trực tiếp K cuộc hội thoại độc lập $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \dots, \mathbf{z}^{(K)}$ (có độ dài tương ứng $L_1, L_2, \dots, L_K$ sao cho $\sum_{k=1}^{K} L_k \le L_{\max}$) nằm liên tiếp nhau trên cùng một hàng dữ liệu:

$$\mathbf{Z}_{\text{packed}} = \Big(\underbrace{z_1^{(1)}, \dots, z_{L_1}^{(1)}}_{\text{Hội thoại 1 (Độ dài } L_1)} , \underbrace{z_1^{(2)}, \dots, z_{L_2}^{(2)}}_{\text{Hội thoại 2 (Độ dài } L_2)} , \dots , \underbrace{z_1^{(K)}, \dots, z_{L_K}^{(K)}}_{\text{Hội thoại } K \text{ (Độ dài } L_K)} \Big). \tag{1.15}$$

Tuy nhiên, để chuỗi đóng gói $\mathbf{Z}_{\text{packed}}$ cho ra kết quả toán học **trùng khít 100%** với việc chạy riêng lẻ từng cuộc hội thoại, hai điều kiện bất biến về Mặt nạ Chú ý và Mã hóa Vị trí bắt buộc phải được thỏa mãn:

1.4.1 Điều kiện 1: Mặt nạ Nhân quả Đường chéo Khối (Block-Diagonal Causal Attention Mask)

Gọi $s_k = \sum_{r=1}^{k-1} L_r$ là chỉ số bắt đầu và $e_k = \sum_{r=1}^{k} L_r$ là chỉ số kết thúc của cuộc hội thoại thứ k trong chuỗi đóng gói (với $s_1 = 0$). Với mọi vị trí toàn cục $i \in \{1, \dots, L_{\text{total}}\}$, ký hiệu $c(i) \in \{1, \dots, K\}$ là chỉ số của cuộc hội thoại chứa token thứ i (tức $s_{c(i)} < i \le e_{c(i)}$).

Trong Pre-training tiêu chuẩn, mặt nạ nhân quả $M^{\text{causal}} \in \{0, -\infty\}^{L \times L}$ chỉ kiểm tra điều kiện $i \ge j$. Nếu dùng M^{causal} cho $\mathbf{Z}_{\text{packed}}$, một token ở vị trí i thuộc Hội thoại 2 ($c(i) = 2$) sẽ có $i > j$ với mọi vị trí j thuộc Hội thoại 1 ($c(j) = 1$), dẫn đến **Rò rỉ Ngữ cảnh (Cross-Document Contamination)**.

Để triệt tiêu hoàn toàn sự giao thoa giữa các hội thoại khác nhau, ta định nghĩa **Mặt nạ Nhân**

quả Đường chéo Khối $\mathbf{M}^{\text{block}} \in \{0, -\infty\}^{L_{\max} \times L_{\max}}$:

$$M_{i,j}^{\text{block}} = \begin{cases} 0 & \text{nếu } i \geq j \textbf{ VÀ } c(i) = c(j) \quad \text{(cùng thuộc một cuộc hội thoại)}, \\ -\infty & \text{nếu } i < j \textbf{ HOẶC } c(i) \neq c(j). \end{cases} \tag{1.16}$$

Khi đưa vào công thức Self-Attention $\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}^{\text{block}}\right)$, với mọi cặp vị trí (i, j) khác cuộc hội thoại ($c(i) \neq c(j)$), trọng số chú ý trở thành:

$$A_{i,j} = \frac{\exp\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} - \infty\right)}{\sum_{r=1}^{L_{\max}} \exp\left(\frac{\mathbf{q}_i^\top \mathbf{k}_r}{\sqrt{d_k}} + M_{i,r}^{\text{block}}\right)} = 0. \tag{1.17}$$

Như vậy, ma trận trọng số chú ý $\mathbf{A}$ có dạng đường chéo khối:

$$\mathbf{A} = \text{diag}\left(\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \ldots, \mathbf{A}^{(K)}\right) = \begin{pmatrix} \mathbf{A}^{(1)}_{L_1 \times L_1} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{A}^{(2)}_{L_2 \times L_2} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{A}^{(K)}_{L_K \times L_K} \end{pmatrix}, \tag{1.18}$$

trong đó mỗi khối con $\mathbf{A}^{(k)} \in \mathbb{R}^{L_k \times L_k}$ là một ma trận tam giác dưới độc lập hoàn toàn.

1.4.2 Điều kiện 2: Đặt lại Chỉ số Vị trí cho Mã hóa Vị trí Quay (RoPE Position Reset)

Các mô hình ngôn ngữ hiện đại sử dụng **Mã hóa Vị trí Quay (Rotary Position Embedding – RoPE)** [60]. Trong RoPE, với một vector Query hoặc Key $\mathbf{u} \in \mathbb{R}^d$ tại vị trí tương đối $p \in \{0, 1, 2, \ldots\}$, không gian $\mathbb{R}^d$ được chia thành $d/2$ mặt phẳng hai chiều trực giao; trên mặt phẳng thứ $m \in \{1, \ldots, d/2\}$, cặp tọa độ (u_{2m-1}, u_{2m}) được quay một góc $p \cdot \theta_m$ (với tần số góc $\theta_m = 10000^{-2(m-1)/d}$) thông qua ma trận quay khối $\mathbf{R}^d_{\Theta,p} \in \mathbb{R}^{d \times d}$:

$$\mathbf{R}^d_{\Theta,p} = \text{diag}\left(\mathbf{R}_1(p), \mathbf{R}_2(p), \ldots, \mathbf{R}_{d/2}(p)\right), \quad \text{với } \mathbf{R}_m(p) = \begin{pmatrix} \cos(p\theta_m) & -\sin(p\theta_m) \\ \sin(p\theta_m) & \cos(p\theta_m) \end{pmatrix}. \tag{1.19}$$

Điểm chú ý giữa vị trí i (mang chỉ số vị trí p_i) và vị trí j (mang chỉ số vị trí p_j) là:

$$\left(\mathbf{R}^d_{\Theta,p_i}\mathbf{q}_i\right)^\top \left(\mathbf{R}^d_{\Theta,p_j}\mathbf{k}_j\right) = \mathbf{q}_i^\top \left(\mathbf{R}^d_{\Theta,p_i}\right)^\top \mathbf{R}^d_{\Theta,p_j}\mathbf{k}_j = \mathbf{q}_i^\top \mathbf{R}^d_{\Theta,p_j-p_i}\mathbf{k}_j. \tag{1.20}$$

Mặc dù điểm chú ý giữa hai token trong cùng một hội thoại chỉ phụ thuộc vào khoảng cách tương đối $p_j - p_i$, nhưng trong thực tế các mô hình còn áp dụng bộ nhớ đệm hoặc có sự nhạy cảm với khoảng giá trị tuyệt đối của p_i trong các phép làm trơn ngoại suy ngữ cảnh (như YaRN / RoPE Scaling trong Llama 3 Dubey et al. 17). Do đó, đối với mọi vị trí toàn cục i thuộc cuộc hội thoại thứ k ($s_k < i \leq e_k$), chỉ số vị trí p_i truyền vào ma trận quay RoPE bắt buộc phải được **đặt lại (reset) về** 0 tại điểm bắt đầu của mỗi cuộc hội thoại:

$$p_i = i - s_{c(i)} - 1 \in \{0, 1, \ldots, L_{c(i)} - 1\}. \tag{1.21}$$

1.5 Quy luật Dữ liệu trong SFT: Giả thuyết LIMA vs. Quy mô Công nghiệp Hiện đại

Câu hỏi chiến lược đặt ra khi thiết kế giai đoạn SFT là: *"Cần bao nhiêu mẫu dữ liệu* (x, y^*) *để căn chỉnh thành công một mô hình nền tảng?"*

1.5.1 Giả thuyết Căn chỉnh Bề mặt (Superficial Alignment Hypothesis – LIMA [73])

Trong công trình **LIMA (*Less Is More for Alignment*)**, Zhou et al. [73] đã phát biểu và kiểm chứng **Giả thuyết Căn chỉnh Bề mặt**:

> *"Tri thức và năng lực của một mô hình ngôn ngữ được học gần như hoàn toàn trong giai đoạn Pre-training. Quá trình căn chỉnh (Alignment) chỉ đơn thuần dạy mô hình nên chọn định dạng con (subdistribution of formats) nào để giao tiếp với người dùng."*

Chỉ với đúng **1,000 mẫu hội thoại** được biên tập thủ công với chất lượng văn phong đồng nhất và độ đa dạng câu hỏi cao (không sử dụng bất kỳ bước RLHF nào), mô hình LIMA-65B đã đạt tỷ lệ thắng ngang ngửa hoặc vượt qua GPT-4 (phiên bản đầu năm 2023) trong 43% các câu hỏi mở do con người đánh giá, vượt xa mô hình Alpaca-65B được huấn luyện trên $52,000$ mẫu dữ liệu chất lượng thấp. Bài học cốt lõi từ LIMA là trong SFT, **chất lượng và độ sạch của phân phối mẫu** y^* **quan trọng hơn số lượng thô gấp nhiều lần**.

1.5.2 Sự Mở rộng Quy mô SFT trong Thực tế Công nghiệp Hiện đại [17, 35]

Tuy nhiên, các nghiên cứu quy mô lớn sau đó như **FLAN** [66], **Llama 3** [17] và **Tülu 3** [35] đã chứng minh rằng: 1,000 mẫu của LIMA chỉ đủ cho các cuộc trò chuyện văn bản mở thông thường. Khi yêu cầu mô hình giải quyết các tác vụ đa kỹ năng khắt khe (giải toán Olympic nhiều bước, viết mã nguồn đa ngôn ngữ, gọi công cụ *Function Calling*, tuân thủ ràng buộc định dạng chính xác và xử lý ngữ cảnh dài), tập dữ liệu SFT hiện đại cần từ **100,000 đến hơn 1,000,000 mẫu chất lượng cao**.

Để tạo ra hàng trăm nghìn mẫu SFT mà vẫn giữ được chất lượng tương đương chuyên gia như LIMA, quy trình công nghiệp hiện đại sử dụng kỹ thuật **Lấy mẫu Bác bỏ kết hợp Chắt lọc Tổng hợp (Rejection Sampling & Synthetic Distillation)** [17]:

1. Với mỗi câu hỏi phức tạp x thu thập từ thực tế, dùng mô hình mạnh nhất hiện có sinh ra N ứng viên trả lời độc lập $\{y^{(1)}, y^{(2)}, \ldots, y^{(N)}\} \sim \pi_{\text{Teacher}}(\cdot \mid x)$ ở nhiệt độ lấy mẫu cao (ví dụ $N \in [16, 64]$).

2. Sử dụng các bộ kiểm tra tự động (như trình biên dịch mã nguồn, bộ giải toán học) kết hợp với Mô hình Phần thưởng (*Reward Model*) để chấm điểm và lọc bỏ toàn bộ các ứng viên sai hoặc trình bày kém, chỉ giữ lại ứng viên đạt điểm tuyệt đối $y^* = \arg\max_{y^{(k)}} r(x, y^{(k)})$ đưa vào tập huấn luyện SFT.

Chương 2

Nhập môn Học Tăng Cường cho LLM, RLHF & Giải thuật PPO

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 2.1. Chương này đóng vai trò cầu nối bản lề trong toàn bộ khóa học: đưa người học từ tư duy Học có Giám sát (*Supervised Learning*) sang tư duy **Học Tăng Cường (Reinforcement Learning – RL)**. Chúng ta sẽ giải thích triệt để ba giới hạn toán học khiến SFT không thể tự giải quyết bài toán căn chỉnh, xây dựng khái niệm RL từ con số 0 dựa trên bài toán dự đoán token tiếp theo, dẫn xuất mô hình đánh giá sở thích Bradley–Terry / Plackett–Luce, phân tích toán học của Định luật Goodhart, chứng minh giải thuật PPO với hàm lợi thế GAE, và giải phẫu bài toán hệ thống 4 mô hình trên phần cứng.

2.1 Ba Giới hạn Toán học và Nhận thức Cốt lõi của SFT

Trước khi tìm hiểu RL, một câu hỏi tự nhiên mà người học xuất phát từ Pre-training luôn đặt ra là: *"Nếu ta đã có tập dữ liệu SFT gồm hàng trăm nghìn mẫu lời giải hoàn hảo của chuyên gia, tại sao mô hình vẫn mắc lỗi ngớ ngẩn và tại sao bắt buộc phải cần đến Học tăng cường?"*

Có ba rào cản nền tảng mà phương pháp Tinh chỉnh có Giám sát không bao giờ có thể tự vượt qua:

2.1.1 1. Học Bắt chước Hành vi Kích hoạt Hiện tượng Ảo giác (Behavior Cloning Triggers Hallucinations)

Trong SFT, mô hình được huấn luyện bằng phương pháp **Bắt chước Hành vi (Behavior Cloning)**: với mỗi câu hỏi x, chuyên gia con người (hoặc một mô hình vượt trội) cung cấp một câu trả lời mẫu $y^* = (y_1^*, \ldots, y_T^*)$ đầy đủ chi tiết, tự tin và chuẩn xác. Hàm mất mát Cross-Entropy ép mô hình:

$$\max_{\theta} \sum_{t=1}^{T} \log \pi_\theta(y_t^* \mid x, y_{<t}^*). \tag{2.1}$$

Tuy nhiên, các nghiên cứu chuyên sâu về tính xác thực của LLM [21, 33] đã chỉ ra một nghịch lý toán học nguy hiểm:

- Gọi $\mathcal{K}_{\text{pre}}$ là tập hợp các tri thức sự kiện đã được mã hóa vào trọng số của mô hình trong giai đoạn Pre-training. Xét một câu hỏi chuyên sâu x thuộc một sự kiện lịch sử hiếm gặp hoặc một định lý trừu tượng nằm ngoài $\mathcal{K}_{\text{pre}}$ ($x \notin \mathcal{K}_{\text{pre}}$).
- Khi đối mặt với x, trạng thái ẩn bên trong của Base Model mang độ bất định rất cao (Entropy của phân phối dự đoán rất lớn).
- Tuy nhiên, trong tập dữ liệu SFT, câu trả lời mẫu y^* vẫn được viết một cách tự tin, khẳng định chắc nịch về ngày tháng, nhân vật và sự kiện.
- Khi ta tối ưu hóa Cross-Entropy trên cặp (x, y^*), gradient cập nhật ép mô hình phải gán xác suất cao cho các khẳng định cụ thể đó. Quy tắc thống kê mà mạng nơ-ron học được không phải là hiểu sâu về sự kiện (vì không có đủ dữ liệu bối cảnh để tạo lập nơ-ron tri thức mới), mà là: **"Khi gặp một câu hỏi mà bên trong mình hoàn toàn mù mờ, hành vi tối ưu để giảm hàm mất mát là sinh ra một câu trả lời khẳng định nghe thật trôi chảy, chuyên nghiệp và thuyết phục!"**

Nói cách khác, như Gekhman et al. [21] đã chứng minh bằng thực nghiệm tại EMNLP 2024, việc ép mô hình học thuộc các câu trả lời vượt quá ranh giới tri thức nội tại trong SFT chính là nguyên nhân trực tiếp kích hoạt và dung dưỡng hiện tượng **Ảo giác (Hallucination)**.

2.1.2 2. Độ lệch Phơi nhiễm (Exposure Bias) và Tích lũy Sai số Bậc hai $\mathcal{O}(T^2)$

Trong SFT, mô hình được huấn luyện dưới chế độ **Teacher Forcing**: tại bước t, để dự đoán từ tiếp theo, mô hình luôn được cung cấp tiền tố chuẩn xác tuyệt đối của chuyên gia $y^*_{<t} = (y^*_1, \ldots, y^*_{t-1})$. Phân phối trạng thái mà mô hình trải nghiệm lúc huấn luyện là:

$$d_{\text{train}}(s_t) = P(x, y^*_{<t}). \tag{2.2}$$

Ngược lại, khi vận hành thực tế (*Inference / Test time*), chuyên gia không còn ở bên cạnh. Mô hình phải tự sinh ra từng token và tự ăn lại chính những token do mình vừa sinh ra $\hat{y}_{<t} = (\hat{y}_1, \ldots, \hat{y}_{t-1}) \sim \pi_\theta$:

$$d_{\text{test}}(s_t) = P(x, \hat{y}_{<t}). \tag{2.3}$$

Định lý 2.1 (Tích lũy Sai số Bậc hai của Bắt chước Hành vi [50, 52])**.** *Giả sử tại mỗi bước sinh token, xác suất mô hình mắc sai sót nhỏ so với chuyên gia bị chặn bởi $\epsilon \in (0, 1)$, tức $\mathbb{E}_{s \sim d_{train}}\left[P_{\pi_\theta}(y_t \neq y^*_t \mid s)\right] \leq \epsilon$. Khi đó, đối với một chuỗi sinh tự hồi quy có độ dài T, tổng kỳ vọng số token sai lệch (hoặc khoảng cách biến thiên toàn phần giữa phân phối sinh của mô hình và phân phối lý tưởng) bị chặn dưới bởi một đại lượng tăng theo cấp số bậc hai của chiều dài chuỗi:*

$$\mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=1}^{T} \mathbb{I}(\hat{y}_t \neq y^*_t)\right] = \mathcal{O}\big(\epsilon \cdot T^2\big). \tag{2.4}$$

Ý tưởng Chứng minh theo Ross et al. [52]. Xét xác suất mô hình rơi ra khỏi quỹ đạo mẫu của chuyên gia tại bước thời gian $k \in \{1, \dots, T\}$. Tại mỗi bước k, với xác suất ϵ, mô hình chọn một token sai lệch $\hat{y}_k \neq y_k^*$. Kể từ bước $k+1$ trở đi cho đến bước cuối cùng T (còn lại $T-k$ bước), trạng thái đầu vào đã bị trôi dạt ra ngoài miền dữ liệu mà SFT từng dạy ($s \notin \text{supp}(d_{\text{train}})$). Do chưa từng được học cách xử lý khi ở ngoài miền phân phối, mô hình có thể tiếp tục mắc lỗi ở mọi bước còn lại. Tổng sai số tích lũy trên toàn bộ chuỗi được tính bằng tổng trên mọi bước k:

$$\sum_{k=1}^{T} \epsilon \cdot (T-k) = \epsilon \left(T^2 - \frac{T(T+1)}{2} \right) = \frac{\epsilon T(T-1)}{2} = \mathcal{O}\big(\epsilon T^2\big). \tag{2.5}$$

□

Hệ quả là trong các chuỗi suy luận dài (ví dụ $T = 4,096$ hoặc $16,384$ tokens), chỉ cần một lỗi nhỏ ở token thứ 50, toàn bộ các token phía sau sẽ sụp đổ dây chuyền do tích lũy sai số bậc hai. SFT hoàn toàn bất lực vì nó không bao giờ dạy mô hình cách **tự phát hiện sai lầm, quay lui (backtracking) và sửa sai** từ chính các trạng thái lỗi do nó gây ra!

2.1.3 3. Sự Bất đối xứng Sinh–Kiểm chứng & Phạt Cứng nhắc ở Mức Token

- **Phạt mù quáng ở mức Token:** Hàm mất mát Cross-Entropy so khớp cơ học từng vị trí với đáp án mẫu y^*. Giả sử đáp án mẫu là: *"Nguyễn Du là tác giả của Truyện Kiều"*. Nếu mô hình viết một câu hoàn toàn chính xác về ngữ nghĩa: *"Truyện Kiều được sáng tác bởi đại thi hào Nguyễn Du"*, hàm mất mát Cross-Entropy sẽ phạt cực nặng trên mọi vị trí vì toàn bộ thứ tự từ bị đảo lộn. Ngược lại, nếu mô hình chỉ đổi đúng một từ: *"Nguyễn Du **không** phải là tác giả của Truyện Kiều"*, câu trả lời làm đảo lộn sự thật nhưng chỉ bị phạt ở đúng một token chữ *"không"*! Cross-Entropy hoàn toàn không có nhận thức về chân giá trị toàn cục của cả câu trả lời.
- **Sự bất đối xứng giữa Sinh tạo và Đánh giá:** Việc yêu cầu con người ngồi viết từ đầu một bài luận xuất sắc hoặc một thuật toán tối ưu (để làm nhãn y^* cho SFT) đòi hỏi chi phí và thời gian rất lớn. Ngược lại, việc cho mô hình sinh ra hai câu trả lời y_1, y_2 rồi hỏi con người: *"Câu nào hay hơn?"* (So sánh cặp – *Pairwise Comparison*) là một nhiệm vụ nhận thức dễ dàng hơn gấp bội đối với con người. SFT hoàn toàn không thể tận dụng được dạng dữ liệu so sánh cặp này.

2.2 Nhập môn Học Tăng Cường (RL) từ Con số 0 dưới Góc nhìn LLM

Để khắc phục ba giới hạn trên, mô hình cần được chuyển sang phương thức học mới: **Học Tăng Cường (Reinforcement Learning – RL)** [61]. Đối với người học mới chỉ biết Pre-training, RL có vẻ xa lạ vì gắn liền với robot hoặc cờ vua. Tuy nhiên, về mặt toán học giải tích, **quá trình một mô hình Transformer sinh văn bản tự hồi quy chính là một Quá trình Quyết định Markov (Markov Decision Process – MDP) hoàn hảo!**

2.2.1 Bảng Ánh xạ Khái niệm Toán học giữa RL và LLM

Hãy nhìn vào sự đối chiếu trực tiếp một-một trong Bảng 2.1.

Khái niệm trong RL Cổ điển	Tương ứng Trực tiếp trong Mô hình Ngôn ngữ Lớn (LLM)
Tác tử (Agent) / Chính sách (π_θ)	Chính là mạng nơ-ron Transformer với bộ trọng số θ.
Không gian Trạng thái ($\mathcal{S}$)	Không gian mọi chuỗi văn bản khả dĩ. Tại bước t, trạng thái $s_t = (x, y_{<t})$ là toàn bộ ngữ cảnh gồm câu hỏi x và các token đã sinh trước đó $y_{<t}$.
Không gian Hành động ($\mathcal{A}$)	Chính là **Bảng từ vựng** V (kích thước $\lvert V\rvert \approx 32,000 - 128,000$).
Hành động tại bước t (a_t)	Việc chọn một token cụ thể $y_t \in V$ từ phân phối xác suất $\pi_\theta(\cdot \mid s_t)$.
Mô hình Chuyển trạng thái $\mathcal{P}(s_{t+1} \mid s_t, a_t)$	**Hoàn toàn tất định (Deterministic):** Trạng thái mới đơn giản là ghép token vừa chọn vào đuôi: $s_{t+1} = (x, y_{<t}, y_t) = (x, y_{\leq t})$. Không có tính ngẫu nhiên của môi trường vật lý!
Quỹ đạo / Một ván chơi (τ)	Toàn bộ quá trình sinh từ token đầu tiên y_1 đến token kết thúc $\langle\lvert\text{im_end}\rvert\rangle$ (y_T), tạo thành cặp câu hỏi–câu trả lời hoàn chỉnh (x, y).
Phần thưởng Toàn cục ($R(\tau)$)	Điểm số vô hướng $r(x, y) \in \mathbb{R}$ đánh giá chất lượng của toàn bộ câu trả lời y đối với câu hỏi x.

Bảng 2.1: Ánh xạ tương đương một-một giữa lý thuyết RL và mô hình ngôn ngữ tự hồi quy.

2.2.2 Dẫn xuất Toán học của Định lý Đạo hàm Chính sách (Policy Gradient Theorem)

Mục tiêu của RL là tìm bộ tham số θ để tối đa hóa kỳ vọng phần thưởng trên toàn bộ các câu trả lời do mô hình tự sinh ra:

$$\mathcal{J}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[r(x,y)\right] = \sum_x P(x) \sum_{y\in V^*} \pi_\theta(y \mid x) r(x,y). \tag{2.6}$$

Một thách thức toán học xuất hiện: trong Phương trình (2.6), tham số θ nằm ở **phân phối lấy mẫu** $\pi_\theta(y \mid x)$ chứ không nằm bên trong hàm phần thưởng $r(x, y)$! Làm thế nào để lấy đạo hàm $\nabla_\theta \mathcal{J}(\theta)$ khi phép lấy mẫu $y \sim \pi_\theta$ là một phép toán rời rạc không khả vi?

Định lý 2.2 (Định lý Đạo hàm Chính sách cho LLM [62, 68])**.** *Với hàm mục tiêu $\mathcal{J}(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r(x,y)]$, đạo hàm theo θ bằng kỳ vọng của tích giữa gradient log-xác suất và phần thưởng:*

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\nabla_\theta \log \pi_\theta(y \mid x) \cdot r(x,y)\right] = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \cdot r(x,y)\right]. \tag{2.7}$$

Chứng minh. Xuất phát từ định nghĩa kỳ vọng dưới dạng tổng rời rạc trên mọi câu trả lời y:

$$\nabla_\theta \mathcal{J}(\theta) = \nabla_\theta \sum_y \pi_\theta(y \mid x) r(x, y) = \sum_y \nabla_\theta \pi_\theta(y \mid x) r(x, y). \tag{2.8}$$

Áp dụng **Bổ đề Đạo hàm Logarit (Likelihood Ratio / Log-Derivative Trick)**: do $\nabla_\theta \log f(\theta) = \frac{\nabla_\theta f(\theta)}{f(\theta)}$, ta có $\nabla_\theta \pi_\theta(y \mid x) = \pi_\theta(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x)$. Thế vào biểu thức trên:

$$\nabla_\theta \mathcal{J}(\theta) = \sum_y \pi_\theta(y \mid x) \left(\nabla_\theta \log \pi_\theta(y \mid x) \cdot r(x, y) \right) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\nabla_\theta \log \pi_\theta(y \mid x) \cdot r(x, y) \right]. \tag{2.9}$$

Mặt khác, theo quy tắc nhân xác suất, $\log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t})$. Do tính tuyến tính của toán tử gradient:

$$\nabla_\theta \log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}). \tag{2.10}$$

Thế biểu thức này vào kỳ vọng, ta thu được phương trình cần chứng minh. □

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 2.2 (So sánh Trực diện: SFT vs. Policy Gradient). Hãy đặt công thức gradient của SFT và RL cạnh nhau trên bảng để thấy sự kết nối tuyệt đẹp:

$$\text{SFT:} \quad \nabla_\theta \mathcal{J}_{\text{SFT}}(\theta) = \sum_{t=1}^{T} (+\mathbf{1}) \cdot \nabla_\theta \log \pi_\theta(y_t^* \mid x, y_{<t}^*), \tag{2.11}$$

$$\text{RL:} \quad \nabla_\theta \mathcal{J}_{\text{RL}}(\theta) = \mathbb{E}_{y \sim \pi_\theta} \left[\sum_{t=1}^{T} \mathbf{r}(\mathbf{x}, \mathbf{y}) \cdot \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \right]. \tag{2.12}$$

Nhận xét mấu chốt: Toán hạng $\nabla_\theta \log \pi_\theta(y_t \mid \dots)$ trong RL **hoàn toàn giống hệt** đạo hàm của hàm Cross-Entropy trong Pre-training và SFT!

- Trong SFT, hệ số nhân luôn cố định là +1: mô hình bị ép tăng xác suất của token mẫu y_t^* bất kể trạng thái bên trong.
- Trong RL, mô hình **tự mình sinh thử** câu trả lời $y \sim \pi_\theta$ (*Rollout*). Hệ số nhân chính là phần thưởng $r(x, y)$:
  - Nếu câu trả lời tự sinh đạt điểm tốt ($r(x, y) > 0$): RL đẩy xác suất các token đó **tăng lên** (tỷ lệ thuận với độ lớn phần thưởng).
  - Nếu câu trả lời tự sinh bị chấm điểm tệ ($r(x, y) < 0$): Dấu âm làm đảo chiều vector gradient – RL **đè xác suất các token đó xuống** để mô hình không bao giờ lặp lại lỗi đó nữa!

2.2.3 Bổ đề Đường Cơ sở Không chệch (Unbiased Baseline Lemma)

Công thức REINFORCE cơ bản ở Phương trình (2.7) có nhược điểm chí mạng là **Phương sai cực lớn (High Variance)**: nếu mọi phần thưởng đều dương (ví dụ $r \in [10, 12]$), thì ngay cả câu trả lời dở nhất ($r = 10$) vẫn được tăng xác suất! Để khắc phục, ta trừ đi một đại lượng đường cơ sở $b(x)$ (*Baseline*).

Bổ đề 2.3 (Đường cơ sở Không làm lệch Gradient)**.** *Với bất kỳ hàm số $b(x)$ nào chỉ phụ thuộc vào câu hỏi x (hoặc hằng số) mà không phụ thuộc vào câu trả lời y, kỳ vọng của gradient khi nhân với $b(x)$ luôn bằng 0 tuyệt đối:*

$$\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\nabla_\theta \log \pi_\theta(y \mid x) \cdot b(x)\right] = \mathbf{0}. \tag{2.13}$$

Do đó, ta có thể tự do trừ $b(x)$ vào phần thưởng mà gradient vẫn không bị chệch (Unbiased):

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\nabla_\theta \log \pi_\theta(y \mid x) \cdot \big(r(x, y) - b(x)\big)\right]. \tag{2.14}$$

Chứng minh. Khai triển kỳ vọng theo tổng rời rạc:

$$\begin{aligned}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\nabla_\theta \log \pi_\theta(y \mid x) \cdot b(x)\right] &= \sum_y \pi_\theta(y \mid x)\nabla_\theta \log \pi_\theta(y \mid x) b(x) \\ &= b(x)\sum_y \nabla_\theta \pi_\theta(y \mid x) = b(x)\nabla_\theta \underbrace{\left(\sum_y \pi_\theta(y \mid x)\right)}_{=1 \text{ (do bảo toàn xác suất)}} = b(x)\nabla_\theta(1) = \mathbf{0}.\end{aligned} \tag{2.15}$$

□

Bằng cách chọn $b(x) = \mathbb{E}_{y\sim\pi_\theta}[r(x, y)]$, hiệu số $r(x, y) - b(x)$ phản ánh chính xác câu trả lời y này **tốt hơn hay tệ hơn mức trung bình** của chính mô hình cho câu hỏi đó, giúp triệt tiêu phương sai mà không làm lệch nghiệm.

2.3 Mô hình hóa Sở thích Con người (Bradley–Terry) & Huấn luyện Reward Model

Trong các câu hỏi mở, không có một công thức toán học định sẵn nào chấm được $r(x, y)$. Ta phải huấn luyện một mạng nơ-ron đại diện cho cảm nhận con người, gọi là **Mô hình Phần thưởng (Reward Model – RM)** $r_\phi(x, y)$ với bộ trọng số ϕ [13, 43, 59].

2.3.1 Tại sao dùng So sánh Cặp (Pairwise Comparison)?

Nếu yêu cầu người đánh giá đọc một câu trả lời và chấm điểm từ 1 đến 10, dữ liệu thu được sẽ cực kỳ nhiễu do **Hiện tượng Trôi dạt Chuẩn độ (Calibration Drift)**: người dễ tính luôn chấm 8–9, người khó tính chỉ chấm 4–5, và cùng một người chấm buổi sáng cũng khác buổi

chiều. Ngược lại, khi đặt hai câu trả lời y_w (*winner / chosen*) và y_l (*loser / rejected*) cạnh nhau và hỏi: *"Câu nào tốt hơn?"* ($y_w \succ y_l \mid x$), độ nhất quán giữa các chuyên gia tăng vọt lên trên 85%.

2.3.2 Mô hình Xác suất Bradley–Terry & Hàm Mất mát của Reward Model

Làm thế nào để ánh xạ tập dữ liệu so sánh cặp $\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$ thành một hàm điểm số thực $r_\phi(x, y) \in \mathbb{R}$?

Định nghĩa 2.1 (Mô hình Bradley–Terry [7])**.** Theo mô hình thống kê Bradley–Terry (1952), với một đối tượng câu hỏi x, xác suất để câu trả lời y_w được ưa thích hơn câu trả lời y_l là hàm Logistic của hiệu số điểm thưởng tiềm ẩn giữa chúng:

$$P(y_w \succ y_l \mid x) = \frac{\exp\big(r_\phi(x, y_w)\big)}{\exp\big(r_\phi(x, y_w)\big) + \exp\big(r_\phi(x, y_l)\big)} = \frac{1}{1 + \exp\big(-(r_\phi(x, y_w) - r_\phi(x, y_l))\big)} = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l\big) \tag{2.16}$$

trong đó $\sigma(z) = \frac{1}{1+e^{-z}}$ là hàm Logistic Sigmoid chuẩn.

Áp dụng nguyên lý Ước lượng Hợp lý Cực đại (MLE) trên toàn bộ tập dữ liệu sở thích $\mathcal{D}_{\text{pref}}$, ta thu được hàm mất mát huấn luyện Reward Model:

$$\mathcal{L}_{\text{RM}}(\phi) = -\,\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pref}}} \Big[\log \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)\Big]. \tag{2.17}$$

Ghi chú Toán học & Hệ thống 2.1 (Tính chất Bất biến Tịnh tiến của Phần thưởng (Shift Invariance)). Quan sát Phương trình (2.16): xác suất sở thích chỉ phụ thuộc vào **hiệu số** $r_\phi(x, y_w) - r_\phi(x, y_l)$. Do đó, nếu ta cộng thêm bất kỳ hàm số $c(x)$ nào vào điểm thưởng:

$$\tilde{r}_\phi(x, y) = r_\phi(x, y) + c(x), \tag{2.18}$$

thì hiệu số $\tilde{r}_\phi(x, y_w) - \tilde{r}_\phi(x, y_l) = \big(r_\phi(x, y_w) + c(x)\big) - \big(r_\phi(x, y_l) + c(x)\big) = r_\phi(x, y_w) - r_\phi(x, y_l)$ hoàn toàn không đổi! Điều này chứng minh rằng **mô hình phần thưởng không có giá trị tuyệt đối duy nhất, mà tồn tại theo một lớp tương đương các hàm số sai khác nhau một hằng số phụ thuộc vào ngữ cảnh** x.

2.3.3 Mở rộng cho So sánh Đa Ứng viên: Mô hình Plackett–Luce [40, 47]

Khi người đánh giá xếp hạng một danh sách gồm $K > 2$ câu trả lời $(y_{(1)} \succ y_{(2)} \succ \cdots \succ y_{(K)} \mid x)$, mô hình Bradley–Terry được tổng quát hóa thành **Mô hình Plackett–Luce**:

$$P\big(y_{(1)} \succ y_{(2)} \succ \cdots \succ y_{(K)} \mid x\big) = \prod_{k=1}^{K-1} \frac{\exp\big(r_\phi(x, y_{(k)})\big)}{\sum_{j=k}^{K} \exp\big(r_\phi(x, y_{(j)})\big)}. \tag{2.19}$$

Hàm mất mát Plackett–Luce thực chất là phân rã xếp hạng thành một chuỗi các bài toán phân loại Softmax đa lớp nối tiếp nhau.

2.3.4 Kiến trúc Transformer của Reward Model: Scalar Value Head tại Token Cuối

Mô hình Reward Model $r_\phi(x, y)$ được khởi tạo từ chính mô hình SFT π_{SFT}:

1. Ta gỡ bỏ ma trận chiếu từ vựng $\mathbf{E}_{\text{out}} \in \mathbb{R}^{|V| \times d}$ ở tầng cuối cùng của Transformer.
2. Thay thế bằng một lớp chiếu tuyến tính vô hướng (**Scalar Value Head**) $\mathbf{w}_r \in \mathbb{R}^{d \times 1}$.
3. **Tại sao chỉ trích xuất điểm thưởng tại vị trí token cuối cùng (EOS token)?** Hãy nhớ lại cơ chế **Causal Self-Attention**: mỗi vị trí token chỉ được phép nhìn về các token đứng trước nó. Tại các token đầu câu, vector ẩn chỉ chứa thông tin cục bộ của phần mở đầu. Duy nhất **token kết thúc câu (EOS token ở vị trí T)** là đã tích hợp đầy đủ thông tin ngữ nghĩa của toàn bộ câu hỏi x và toàn bộ câu trả lời $y = (y_1, \ldots, y_T)$ qua tất cả các tầng chú ý. Do đó, điểm thưởng vô hướng cho toàn bộ câu trả lời được tính bằng:
$$r_\phi(x, y) = \mathbf{w}_r^\top \mathbf{h}_T \in \mathbb{R}. \tag{2.20}$$

2.3.5 RLAIF & Constitutional AI [6, 36]

Khi quy mô mô hình vượt quá hàng chục tỷ tham số, chi phí thuê con người gán nhãn hàng triệu cặp so sánh trở thành nút thắt cổ chai. Phương pháp **RLAIF (Reinforcement Learning from AI Feedback)** và **Constitutional AI** của Anthropic [6] sử dụng một mô hình ngôn ngữ lớn mạnh (như Claude hoặc GPT-4) đóng vai trò giám khảo:

- Mô hình giám khảo được cung cấp một **Bộ Hiến pháp (Constitution)** gồm các nguyên tắc đạo đức và quy tắc ứng xử rõ ràng.
- Giám khảo AI tự động phân tích và chọn $y_w \succ y_l$, tạo ra tập dữ liệu sở thích tổng hợp khổng lồ.
- Lee et al. [36] tại Google đã chứng minh RLAIF đạt hiệu năng căn chỉnh tương đương hoặc thậm chí vượt trội RLHF truyền thống nhờ tính nhất quán cao và không bị mệt mỏi nhận thức.

2.4 Tối ưu hóa Chính sách với Ràng buộc KL, Định luật Goodhart & PPO

Khi đã có Reward Model $r_\phi(x, y)$ đóng băng, ta huấn luyện mô hình ngôn ngữ (lúc này đóng vai trò là **Policy / Actor** π_θ, khởi tạo từ π_{SFT}) để tối đa hóa điểm thưởng.

2.4.1 Định luật Goodhart & Quy luật Quá Tối ưu hóa Phần thưởng (Reward Model Overoptimization Gao et al. 20)

Nếu ta giải bài toán tối ưu hóa ngây thơ:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[r_\phi(x, y) \right], \tag{2.21}$$

thì hệ thống sẽ sụp đổ theo **Định luật Goodhart**: *"Khi một thước đo trở thành mục tiêu tối ưu hóa, nó không còn là một thước đo tốt nữa."*

Mô hình phần thưởng r_ϕ chỉ là một mạng nơ-ron xấp xỉ trên một tập dữ liệu hữu hạn; nó luôn có các lỗ hổng toán học ở những vùng không gian dữ liệu ngoài phân phối (*Out-of-Distribution*). Trong nghiên cứu kinh điển về Quy luật Quá tối ưu hóa, Gao et al. [20] đã chứng minh bằng thực nghiệm:

- Khi π_θ được tối ưu hóa quá mức trên r_ϕ (*Proxy Reward*), điểm số mà r_ϕ chấm cho câu trả lời tiếp tục tăng tuyến tính lên trời.
- Tuy nhiên, chất lượng thực tế dưới con mắt của con người (*Gold / True Reward*) chỉ tăng ở giai đoạn đầu, sau đó **lao dốc không phanh**!
- Mô hình khai thác lỗ hổng của r_ϕ bằng cách sinh ra các văn bản bất thường: lặp lại từ khóa nịnh bợ, kéo dài gạch đầu dòng vô tận, hoặc xuất ra chuỗi ký tự vô nghĩa nhưng vô tình kích hoạt các nơ-ron trọng số dương cực lớn trong r_ϕ (**Reward Hacking**).

2.4.2 Hàm Mục tiêu RLHF với Ràng buộc Phân kỳ KL

Để ngăn chặn Reward Hacking, công thức RLHF chuẩn mực [43, 59, 75] bắt buộc phải trừ đi một đại lượng phạt dựa trên **Phân kỳ Kullback–Leibler ($\mathbb{D}_{\mathrm{KL}}$)** giữa chính sách đang huấn luyện π_θ và mô hình tham chiếu đóng băng π_{ref} (chính là bản sao của π_{SFT} ban đầu):

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}} \left[\mathbb{E}_{y\sim\pi_\theta(\cdot|x)} \left[r_\phi(x, y) \right] - \beta\, \mathbb{D}_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x) \right) \right], \tag{2.22}$$

trong đó $\beta > 0$ là hệ số phạt KL, đóng vai trò như độ cứng của một sợi dây lò xo giữ cho π_θ không đi quá xa khỏi không gian ngôn ngữ tự nhiên của π_{ref}.

Theo định nghĩa của phân kỳ KL trên chuỗi tự hồi quy:

$$\mathbb{D}_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x) \right) = \mathbb{E}_{y\sim\pi_\theta} \left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} \right] = \mathbb{E}_{y\sim\pi_\theta} \left[\sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})} \right]. \tag{2.23}$$

Do đó, ta có thể phân rã mức phạt KL vào từng bước thời gian t, tạo thành **Phần thưởng**

Tổng hợp tại Bước t ($\tilde{r}_t$):

$$\tilde{r}_t = \begin{cases} -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} & \text{với } 1 \le t < T \quad \text{(Chỉ nhận phạt độ lệch KL)}, \\ r_\phi(x, y) - \beta \log \dfrac{\pi_\theta(y_T \mid x, y_{<T})}{\pi_{\text{ref}}(y_T \mid x, y_{<T})} & \text{tại } t = T \quad \text{(Cộng thêm điểm thưởng toàn cục } r_\phi \text{ ở token cuối)}. \end{cases} \tag{2.24}$$

2.5 Giải thuật PPO & Generalized Advantage Estimation (GAE)

2.5.1 Bài toán Quy kết Công trạng & Mạng Phê bình (Critic V_ψ)

Quan sát Phương trình (2.24): điểm thưởng thực sự $r_\phi(x, y)$ chỉ xuất hiện ở bước cuối cùng $t = T$. Trong một chuỗi dài 500 từ, nếu 450 từ đầu rất hay nhưng 50 từ cuối bị sai làm tổng điểm âm, làm thế nào để biết chính xác token nào có công, token nào có tội? Đây chính là **Bài toán Quy kết Công trạng (Credit Assignment Problem)**.

Để giải quyết, giải thuật PPO đưa vào một mạng nơ-ron thứ hai có thể huấn luyện được: **Mạng Phê bình / Mô hình Giá trị (Critic / Value Model V_ψ)** với tham số ψ. Khác với r_ϕ (chỉ xuất 1 điểm ở cuối câu), Critic $V_\psi(s_t) = V_\psi(x, y_{<t}) \in \mathbb{R}$ xuất ra một giá trị tại **mọi vị trí token** $t = 1, \ldots, T$, ước lượng tổng phần thưởng kỳ vọng thu được từ trạng thái đó cho đến hết chuỗi:

$$V^\pi(s_t) = \mathbb{E}_\pi \left[\sum_{l=0}^{T-t} \gamma^l \tilde{r}_{t+l} \,\middle|\, s_t \right]. \tag{2.25}$$

Sai số dự đoán theo thời gian (**Temporal-Difference Residual – TD Error**) tại bước t được định nghĩa là:

$$\delta_t^V = \tilde{r}_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t). \tag{2.26}$$

Ý nghĩa trực quan: Trước khi sinh token y_t, Critic dự đoán triển vọng câu trả lời đạt 5 điểm ($V(s_t) = 5$). Sau khi Actor vừa chọn xong token y_t, triển vọng vọt lên 8 điểm ($V(s_{t+1}) = 8$). Vậy chính hành động chọn y_t đã tạo ra giá trị gia tăng $\delta_t^V \approx +3 > 0$!

2.5.2 Ước lượng Lợi thế Tổng quát (GAE – Schulman et al., 2016)

Định nghĩa 2.2 (Ước lượng Lợi thế GAE [54])**.** Hàm Lợi thế A_t đo lường hành động y_t tốt hơn mức trung bình của trạng thái s_t bao nhiêu. Bộ ước lượng GAE(γ, λ) lấy tổng trung bình có trọng số mũ suy giảm của các sai số TD:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}^V = \delta_t^V + (\gamma\lambda)\delta_{t+1}^V + (\gamma\lambda)^2 \delta_{t+2}^V + \ldots \tag{2.27}$$

Định lý 2.4 (Sự Cân bằng Phương sai và Độ chệch của GAE [54])**.** *Tham số $\lambda \in [0, 1]$ điều chỉnh sự đánh đổi toán học giữa phương sai và độ chệch:*

*(a) Khi $\lambda = 0$: $\hat{A}_t^{GAE} = \delta_t^V = \tilde{r}_t + \gamma V(s_{t+1}) - V(s_t)$. Đây là bộ ước lượng TD(0): có **phương***

sai nhỏ nhất *(chỉ phụ thuộc vào 1 bước ngẫu nhiên), nhưng có* ***độ chệch lớn nhất*** *vì phụ thuộc hoàn toàn vào độ chính xác của mạng Critic* V_ψ.

(b) Khi $\lambda = 1$*:* $\hat{A}_t^{GAE} = \sum_{l=0}^{T-t} \gamma^l \tilde{r}_{t+l} - V(s_t)$*. Đây là bộ ước lượng Monte Carlo:* ***hoàn toàn không chệch (Unbiased)****, nhưng có* ***phương sai lớn nhất*** *vì là tổng của hàng trăm biến ngẫu nhiên dọc theo toàn bộ chuỗi sinh.*

(c) Trong thực tế huấn luyện LLM, người ta thường chọn $\lambda \in [0.95, 0.98]$ *và* $\gamma = 1$ *để dung hòa tối ưu giữa hai thái cực.*

2.5.3 Hàm Mục tiêu Cắt ngưỡng của PPO (PPO-Clip Schulman et al. 55)

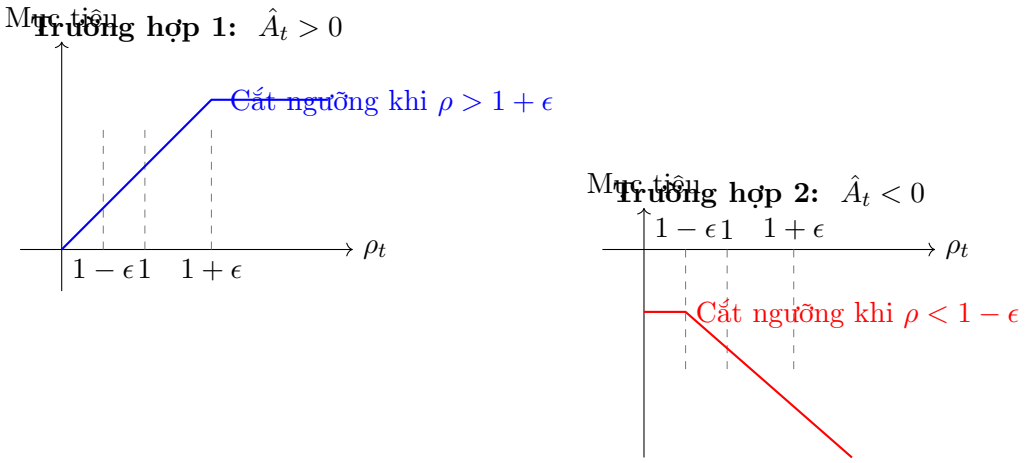
Trong quá trình huấn luyện, ta muốn dùng lại các quỹ đạo vừa sinh ra để cập nhật mạng π_θ qua nhiều bước gradient. Gọi $\pi_{\theta_{\text{old}}}$ là chính sách tại thời điểm sinh dữ liệu, tỷ số thay đổi xác suất (*Importance Sampling Ratio*) tại bước t là:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}. \tag{2.28}$$

Nếu tối ưu hóa trực tiếp $\rho_t(\theta)\hat{A}_t$, một bước gradient quá đà có thể làm $\rho_t(\theta)$ tăng vọt, phá hủy hoàn toàn phân phối ngôn ngữ. Thuật toán **PPO-Clip** [55] đưa ra hàm chặn ngưỡng an toàn:

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta)\hat{A}_t,\ \text{clip}\left(\rho_t(\theta),\, 1-\epsilon,\, 1+\epsilon\right)\hat{A}_t \right) \right], \tag{2.29}$$

với $\epsilon \approx 0.2$ (chỉ cho phép xác suất dịch chuyển tối đa $\pm 20\%$).



Hình 2.1: Cơ chế bảo vệ của hàm cắt ngưỡng PPO-Clip đối với lợi thế dương và lợi thế âm.

Phân tích toán học cơ chế bảo vệ trong Hình 2.1:

- **Khi** $\hat{A}_t > 0$ **(Hành động tốt):** Ta muốn tăng $\rho_t(\theta)$. Khi $\rho_t > 1+\epsilon$, giá trị bị chặn ở $(1+\epsilon)\hat{A}_t$, làm đạo hàm bằng 0 tuyệt đối, ngăn không cho mô hình tham lam cập nhật quá đà trên một mẫu đơn lẻ.

- **Khi $\hat{A}_t < 0$ (Hành động tệ):** Ta muốn giảm $\rho_t(\theta)$. Khi $\rho_t < 1 - \epsilon$, giá trị bị chặn ở $(1 - \epsilon)\hat{A}_t$, đạo hàm triệt tiêu về 0, ngăn không cho mô hình đè xác suất xuống quá sâu gây sụp đổ phân phối.

2.6 Kiến trúc Hệ thống 4 Mô hình trong PPO & Xung đột Phần cứng

Mặc dù giải thuật PPO rất chặt chẽ về mặt toán học, việc vận hành nó trong thực tế kỹ thuật lại là một thách thức khổng lồ [57].

2.6.1 Bài toán 4 Mô hình trên VRAM

Trong khi SFT chỉ cần duy trì **1 mô hình** trên bộ nhớ, một vòng lặp huấn luyện PPO tiêu chuẩn đòi hỏi phải nạp và phối hợp đồng thời **4 mô hình lớn** (Bảng 2.2).

Mô hình	**Ký hiệu**	Trạng thái Trọng số	Chi phí Bộ nhớ Phần cứng (VRAM)
1. Actor (Policy)	π_θ	**Trainable** (Cập nhật đạo hàm)	**Cực nặng:** Chiếm đầy đủ Trọng số (Weights), Đạo hàm (Gradients) và Trạng thái bộ tối ưu AdamW (FP32 Master Weights + 2 Moments).
2. Critic (Value)	V_ψ	**Trainable** (Cập nhật đạo hàm)	**Cực nặng:** Kích thước tương đương Actor, tốn đầy đủ Weights, Gradients và AdamW Optimizer States.
3. Reference Model	$\pi_{\mathbf{ref}}$	**Frozen** (Đóng băng hoàn toàn)	**Trung bình:** Chỉ cần lưu Trọng số và chạy lan truyền xuôi (Forward pass) để tính tỷ số KL tại từng token.
4. Reward Model	r_ϕ	**Frozen** (Đóng băng hoàn toàn)	**Trung bình:** Chỉ cần lưu Trọng số và chạy lan truyền xuôi để chấm điểm toàn cục ở token cuối cùng.

Bảng 2.2: Bảng phân giải 4 mô hình đồng thời trong vòng lặp huấn luyện PPO truyền thống.

Với mô hình kích thước 70 tỷ tham số (70B), việc duy trì 4 mô hình đồng thời (trong đó 2 mô hình có đầy đủ trạng thái AdamW) đòi hỏi một cụm máy chủ gồm hàng chục GPU cao cấp nhất chỉ để chứa vừa tham số tĩnh, chưa kể bộ nhớ dành cho Activations và KV-Cache!

2.6.2 Sự Xung đột Bản chất Phần cứng giữa Hai Pha trong Vòng lặp PPO

Ngoài dung lượng VRAM, hệ thống PPO bị giằng xé bởi sự xung đột về bản chất tính toán giữa hai pha trong mỗi vòng lặp:

1. **Pha 1: Sinh văn bản Tự hồi quy (Generation / Rollout Phase):** Actor π_θ phải tự sinh câu trả lời từng token một ($y_1 \to y_2 \to \cdots \to y_T$). Tại mỗi bước sinh 1 token, GPU phải tải toàn bộ hàng chục Gigabyte trọng số từ bộ nhớ toàn cục (HBM) vào nhân tính toán chỉ để thực hiện phép nhân ma trận–vector rất nhỏ. Quá trình này bị nghẽn

băng thông bộ nhớ (**Memory-Bandwidth Bound**), khiến các nhân tính toán ma trận Tensor Core hoạt động dưới 15% công suất lý thuyết, nhưng lại **chiếm tới** $75\% - 85\%$ **tổng thời gian của cả vòng lặp PPO**!

2. **Pha 2: Huấn luyện & Lan truyền Ngược (Training Phase):** Khi đã thu thập xong toàn bộ chuỗi (x, y), cả 4 mô hình có thể xử lý song song tất cả các token cùng lúc bằng các phép nhân ma trận–ma trận lớn. Pha này bị giới hạn bởi năng lực tính toán (**Compute-Bound**) và đòi hỏi dung lượng bộ nhớ cực lớn để lưu các ma trận kích hoạt trung gian phục vụ lan truyền ngược.

Chính vì sự phức tạp tột độ của bài toán 4 mô hình, tính bất ổn định của mạng Critic và nút thắt cổ chai phần cứng của PPO, cộng đồng nghiên cứu AI đã đặt câu hỏi: **"Liệu có con đường toán học nào đi thẳng từ dữ liệu sở thích sang chính sách tối ưu mà không cần huấn luyện Reward Model, không cần mạng Critic, và không cần chạy vòng lặp sinh mẫu RL hay không?"** Câu trả lời lịch sử chính là **DPO**.

Chương 3

Giải phẫu Toán học Direct Preference Optimization (DPO) & Các Biến thể

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 3.1. Chương này dành trọn thời lượng để giải phẫu một trong những đột phá toán học thanh lịch và có tầm ảnh hưởng sâu rộng nhất trong lịch sử Post-training: công trình **Direct Preference Optimization (DPO)** của nhóm tác giả Stanford [48]. Người học sẽ được dẫn dắt từng bước biến đổi giải tích để hiểu tại sao ta có thể loại bỏ hoàn toàn Reward Model và vòng lặp PPO, phân tích động lực học của gradient DPO, nhận diện 4 cạm bẫy toán học của DPO gốc và tìm hiểu các biến thể cải tiến hiện đại (**SimPO**, **ORPO**, **IPO**, **KTO**, **RPO**).

3.1 Dẫn xuất Toán học Toàn vẹn của DPO (Rafailov et al., 2023)

Mục tiêu Toán học: Giải bài toán cực đại hóa phần thưởng có ràng buộc phân kỳ KL trong RLHF bằng phương pháp giải tích để tìm ra mối liên hệ hàm số tất yếu giữa mô hình phần thưởng tiềm ẩn $r(x, y)$ và chính sách ngôn ngữ $\pi_\theta(y \mid x)$, từ đó thay thế hoàn toàn vòng lặp PPO bằng một hàm mất mát giải tích đơn giản.

3.1.1 Bước 1: Tìm Nghiệm Đóng (Closed-form Solution) của Bài toán RL có Ràng buộc KL

Xét bài toán tối ưu hóa RLHF tại một câu hỏi $x \sim \mathcal{D}$ với một hàm phần thưởng $r(x, y)$ bất kỳ:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} \left[r(x, y) \right] - \beta \, \mathbb{D}_{\mathrm{KL}} \Big(\pi(\cdot \mid x) \parallel \pi_{\mathrm{ref}}(\cdot \mid x) \Big). \tag{3.1}$$

Viết tường minh định nghĩa của kỳ vọng và phân kỳ KL dưới dạng tổng rời rạc trên toàn bộ

không gian chuỗi câu trả lời $y \in V^*$:

$$\max_{\pi} \sum_{y} \pi(y \mid x) r(x, y) - \beta \sum_{y} \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{3.2}$$

Đổi dấu toàn bộ biểu thức (chuyển từ bài toán cực đại hóa max sang cực tiểu hóa min) và chia cho hằng số dương $\beta > 0$:

$$\min_{\pi} \sum_{y} \pi(y \mid x) \left(\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \frac{1}{\beta} r(x, y) \right). \tag{3.3}$$

Sử dụng tính chất logarit $\frac{1}{\beta} r(x, y) = \log \exp\left(\frac{1}{\beta} r(x, y)\right)$ và quy tắc hiệu logarit $\log a - \log b = \log \frac{a}{b}$:

$$\min_{\pi} \sum_{y} \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right). \tag{3.4}$$

Mẫu số $\pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ trong Phương trình (3.4) chưa phải là một phân phối xác suất hợp lệ vì tổng của nó theo mọi y chưa chắc bằng 1. Ta định nghĩa **Hàm phân hoạch (Partition Function)** $Z(x)$ làm hằng số chuẩn hóa:

$$Z(x) = \sum_{y} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y) \right). \tag{3.5}$$

Ghi chú Toán học & Hệ thống 3.1. Lưu ý cực kỳ quan trọng: $Z(x)$ là tổng lấy theo mọi câu trả lời y, do đó $Z(x)$ là một hàm số chỉ phụ thuộc vào câu hỏi x và mô hình gốc π_{ref}, hoàn toàn **độc lập và không phụ thuộc vào chính sách π đang cần tìm**.

Chia và nhân mẫu số với $Z(x)$, đồng thời tách $\log Z(x)$ ra ngoài dấu tổng (vì $\sum_y \pi(y \mid x) = 1$):

$$\begin{aligned}
&\min_{\pi} \sum_{y} \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right) \cdot Z(x)} \right) \\
\iff &\min_{\pi} \left[\sum_{y} \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right) - \sum_{y} \pi(y \mid x) \log Z(x) \right] \\
\iff &\min_{\pi} \left[\mathbb{D}_{\text{KL}} \left(\pi(\cdot \mid x) \parallel \pi^*(\cdot \mid x) \right) - \log Z(x) \right],
\end{aligned} \tag{3.6}$$

trong đó phân phối xác suất hợp lệ dưới mẫu số được đặt tên là $\pi^*(y \mid x)$:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y) \right). \tag{3.7}$$

Định lý 3.1 (Nghiệm Đóng Toàn cục của Bài toán RL có Ràng buộc KL)**.** *Vì số hạng $\log Z(x)$ trong Phương trình (3.6) không phụ thuộc vào π, và theo Bất đẳng thức Gibbs trong lý thuyết thông tin, phân kỳ KL luôn không âm ($\mathbb{D}_{\text{KL}} \geq 0$) và đạt giá trị cực tiểu bằng 0 khi và chỉ khi hai phân phối trùng nhau hoàn toàn, nghiệm tối ưu toàn cục duy nhất $\pi^*(y \mid x)$ của bài toán*

RLHF bắt buộc phải có dạng phân phối Gibbs/Boltzmann:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{ref}(y \mid x)\exp\left(\frac{1}{\beta}r(x,y)\right). \tag{3.8}$$

3.1.2 Bước 2: Phép Đổi biến Đột phá – Biểu diễn Ngược Hàm Phần thưởng theo Chính sách Tối ưu

Từ nghiệm đóng ở Theorem 3.1, ta biến đổi đại số để rút hàm phần thưởng $r(x,y)$ ra một vế:

$$\frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} = \frac{1}{Z(x)}\exp\left(\frac{1}{\beta}r(x,y)\right) \iff \exp\left(\frac{1}{\beta}r(x,y)\right) = Z(x)\frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{3.9}$$

Lấy logarit tự nhiên hai vế và nhân với β:

$$r(x,y) = \beta\log\frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta\log Z(x). \tag{3.10}$$

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 3.2 (Bản chất Triết học: Mọi Mô hình Ngôn ngữ Thực chất là một Reward Model Tiềm ẩn). Phương trình (3.10) là phát hiện trung tâm của bài báo Rafailov et al. [48]: **Có sự tương ứng một-một giữa hàm phần thưởng $r(x,y)$ và chính sách tối ưu $\pi^*(y \mid x)$**. Thay vì phải dùng một mạng nơ-ron riêng biệt để xấp xỉ $r_\phi(x,y)$ rồi dùng thuật toán RL đi tìm π_θ, ta có thể trực tiếp tham số hóa chính sách ngôn ngữ $\pi_\theta(y \mid x)$ và dùng chính nó để định nghĩa hàm phần thưởng tiềm ẩn!

3.1.3 Bước 3: Thế vào Mô hình Bradley–Terry & Sự Triệt tiêu Kỳ diệu của Hàm Phân hoạch $Z(x)$

Nếu đứng riêng một mình, biểu thức $r(x,y)$ ở Phương trình (3.10) là bất khả thi để tính toán trên máy tính vì chứa $\log Z(x) = \log\sum_y \pi_{\text{ref}}(y \mid x)\exp\left(\frac{1}{\beta}r(x,y)\right)$ – muốn tính $Z(x)$ ta phải tính tổng qua toàn bộ $|V|^T$ câu trả lời có thể tồn tại trong vũ trụ ngôn ngữ!

Tuy nhiên, hãy nhớ lại mô hình sở thích Bradley–Terry (Phương trình (2.16) ở Chương 2): xác suất để câu trả lời tốt y_w thắng câu trả lời dở y_l cho cùng một câu hỏi x chỉ phụ thuộc vào **hiệu số giữa hai điểm thưởng** $r(x,y_w) - r(x,y_l)$. Khi ta thay Phương trình (3.10) vào hiệu số này:

$$\begin{aligned} r(x,y_w) - r(x,y_l) &= \left(\beta\log\frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta\log Z(x)\right) - \left(\beta\log\frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta\log Z(x)\right) \\ &= \beta\log\frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \underbrace{\big(\beta\log Z(x) - \beta\log Z(x)\big)}_{=0 \text{ (Triệt tiêu hoàn toàn!)}}. \end{aligned} \tag{3.11}$$

Hai số hạng bất khả thi $\beta\log Z(x)$ (vốn chỉ phụ thuộc vào câu hỏi x) **triệt tiêu lẫn nhau hoàn toàn**!

Thay hiệu số vừa tìm được vào mô hình xác suất Bradley–Terry $P(y_w \succ y_l \mid x) = \sigma\big(r(x,y_w) - r(x,y_l)\big)$, và thay chính sách tối ưu π^* bằng mạng nơ-ron chính sách cần huấn luyện π_θ, ta thu

được xác suất sở thích biểu diễn trực tiếp qua mô hình ngôn ngữ:

$$P_\theta(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right). \tag{3.12}$$

Áp dụng hàm mất mát Negative Log-Likelihood trên tập dữ liệu sở thích $\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$, ta thu được **Hàm Mất Mát DPO ($\mathcal{L}_{\text{DPO}}$)**:

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]} \tag{3.13}$$

Ghi chú Toán học & Hệ thống 3.2 (Tính toán Log-Probability trên Toàn bộ Chuỗi). Trong Phương trình (3.13), đại lượng $\log \pi_\theta(y \mid x)$ được tính bằng tổng log-probability của từng token thuộc câu trả lời:

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}). \tag{3.14}$$

Phép tính này được thực hiện bằng một lượt lan truyền xuôi duy nhất qua mạng Transformer với mặt nạ Prompt Masking giống hệt SFT (Chương 1).

3.2 Giải phẫu Động lực học Gradient của DPO

Để hiểu sâu sắc cơ chế ổn định và tự điều tốc của DPO, hãy định nghĩa **Điểm thưởng Tiềm ẩn (Implicit Reward)** do chính sách π_θ tự định nghĩa là:

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{3.15}$$

Khi đó đối số bên trong hàm Sigmoid của DPO là hiệu số phần thưởng tiềm ẩn: $\Delta\hat{r}_\theta = \hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)$.

Định lý 3.2 (Công thức Gradient của DPO [48])**.** *Đạo hàm của hàm mất mát $\mathcal{L}_{DPO}(\theta)$ theo tham số θ của mạng nơ-ron có dạng:*

$$\nabla_\theta \mathcal{L}_{DPO}(\theta) = -\beta\, \mathbb{E}_{(x,y_w,y_l)}\left[\underbrace{\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)}_{\text{(1) Trọng số Thích ứng Động}} \cdot \left(\underbrace{\nabla_\theta \log \pi_\theta(y_w \mid x)}_{\text{(2) Kéo tăng } y_w} - \underbrace{\nabla_\theta \log \pi_\theta(y_l \mid x)}_{\text{(3) Đè giảm } y_l}\right)\right]. \tag{3.16}$$

Chứng minh. Sử dụng quy tắc đạo hàm hàm hợp cho $\ell = -\log \sigma(u)$ với $u = \beta\big(\log \pi_\theta(y_w \mid x) - \log \pi_{\text{ref}}(y_w \mid x)\big) - \beta\big(\log \pi_\theta(y_l \mid x) - \log \pi_{\text{ref}}(y_l \mid x)\big)$. Ta có $\frac{d}{du}[-\log \sigma(u)] = -\frac{\sigma'(u)}{\sigma(u)} = -\frac{\sigma(u)(1-\sigma(u))}{\sigma(u)} = -(1 - \sigma(u)) = -\sigma(-u)$. Do đó:

$$\nabla_\theta \ell = -\sigma(-u)\nabla_\theta u = -\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big) \cdot \beta\left(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)\right). \tag{3.17}$$

□

3.2.1 Ý nghĩa Cơ học của Ba Thành phần trong Gradient DPO

1. **Thành phần (2)** $+\nabla_\theta \log \pi_\theta(y_w \mid x)$**:** Hoạt động giống hệt đạo hàm của SFT – tạo một lực kéo gradient làm **tăng xác suất** sinh ra câu trả lời được con người ưa chuộng y_w.

2. **Thành phần (3)** $-\nabla_\theta \log \pi_\theta(y_l \mid x)$**:** Tạo một lực đẩy ngược chiều làm **giảm xác suất** sinh ra câu trả lời bị từ chối y_l.

3. **Thành phần (1)** $\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)$ **– "Bộ Điều tốc Tự động":** Trước khi DPO ra đời, các phương pháp loại bỏ câu dở ngây thơ (*Unlikelihood Training*) lấy thẳng hiệu số $\nabla \log \pi(y_w) - \nabla \log \pi(y_l)$ không có trọng số điều chỉnh. Khi đó mô hình nhanh chóng bị sụp đổ vì nó tiếp tục đè xác suất của y_l xuống $-\infty$ ngay cả khi câu đó đã bị lãng quên! Trong DPO, trọng số (1) đo lường **mức độ đánh giá sai hiện tại của mô hình**:

   - **Khi mô hình đang đánh giá sai nghiêm trọng** (chấm câu dở y_l cao điểm hơn câu tốt y_w, tức $\hat{r}_\theta(y_l) > \hat{r}_\theta(y_w)$): Đối số của hàm σ là số dương $\implies \sigma(\cdot) \to 1$. DPO sinh ra **lực cập nhật gradient cực đại** để sửa sai.
   - **Khi mô hình đã phân biệt đúng với biên độ an toàn** ($\hat{r}_\theta(y_w) \gg \hat{r}_\theta(y_l)$): Đối số $\hat{r}_\theta(y_l) - \hat{r}_\theta(y_w)$ rất âm $\implies \sigma(\cdot) \to 0$. Gradient tự động triệt tiêu về $\mathbf{0}$, **mô hình tự động ngừng cập nhật trên cặp mẫu đó** để bảo toàn phân phối ngôn ngữ tự nhiên của π_{ref}!

3.3 Bốn Cạm bẫy Lý thuyết của DPO Gốc & Các Biến thể Khắc phục

Dù DPO giúp cắt giảm quy trình từ 4 mô hình xuống còn 2 mô hình (π_θ và π_{ref}), các nghiên cứu giai đoạn 2024–2025 đã vạch ra 4 cạm bẫy toán học lớn của DPO gốc và đề xuất các phương pháp giải quyết triệt để:

3.3.1 Cạm bẫy 1: Thiên kiến Độ dài (Length / Verbosity Bias) & Giải pháp SimPO [41, 46]

- **Nguyên nhân toán học:** Phần thưởng tiềm ẩn của DPO được định nghĩa bằng tổng log-prob trên toàn bộ độ dài câu:

$$\hat{r}_\theta(x, y) = \beta \sum_{t=1}^{|y|} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}. \tag{3.18}$$

  Giả sử tại mỗi token, mô hình tạo ra một độ lệch log-ratio dương nhỏ $\epsilon = +0.05$. Một câu trả lời ngắn gọn súc tích dài $|y| = 100$ tokens chỉ tích lũy được phần thưởng $\hat{r} = 100 \times 0.05 = 5$. Ngược lại, một câu trả lời dài dòng lan man $|y| = 1,000$ tokens tích lũy được phần thưởng lên tới $\hat{r} = 1,000 \times 0.05 = 50$! DPO vô tình thưởng cho các câu trả lời dài lê thê (**Verbosity Bias**), khiến mô hình sau DPO mắc bệnh nói nhiều nhưng rỗng nội dung.

Định nghĩa 3.1 (Thuật toán SimPO – Meng, Xia & Chen, NeurIPS 2024 [41])**.** Phương pháp **SimPO (Simple Preference Optimization)** giải quyết triệt để thiên kiến độ dài bằng hai cải tiến toán học:

(a) **Chuẩn hóa theo chiều dài (Length Normalization):** Chia trực tiếp tổng log-probability cho độ dài chuỗi $|y|$ để lấy giá trị trung bình trên mỗi token:

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_\theta(y \mid x) = \frac{\beta}{|y|} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}). \tag{3.19}$$

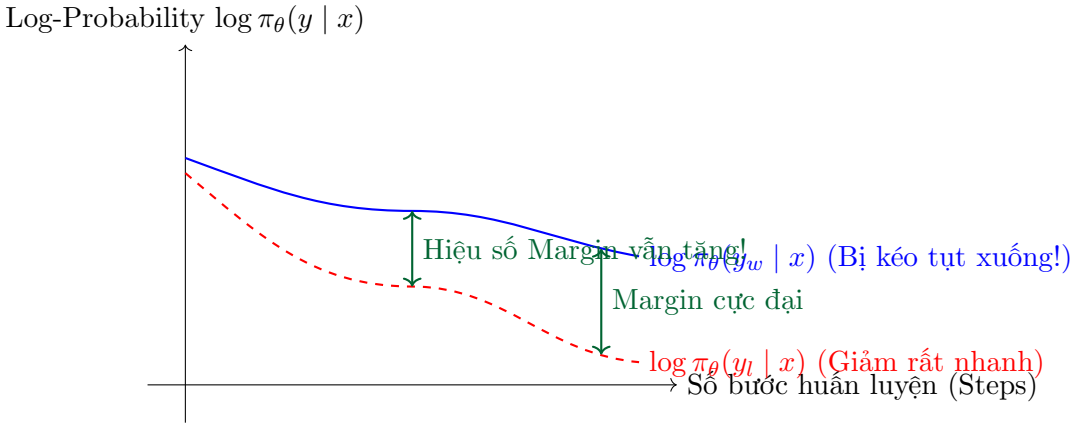
(b) **Loại bỏ Mô hình Tham chiếu & Đặt Biên độ Thắng Mục tiêu $\gamma > 0$:** SimPO loại bỏ hoàn toàn π_{ref}, đồng thời đưa thêm một hằng số mục tiêu $\gamma > 0$ (*Target Reward Margin*) vào mô hình Bradley–Terry:

$$\boxed{\mathcal{L}_{\text{SimPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma \right) \right]} \tag{3.20}$$

Nhờ đó, SimPO chỉ cần duy trì **duy nhất 1 mô hình trên GPU**, loại bỏ hoàn toàn bộ nhớ cho π_{ref}, triệt tiêu thiên kiến độ dài và vượt trội DPO trên AlpacaEval 2 và MT-Bench.

3.3.2 Cạm bẫy 2: Hiện tượng Dịch chuyển Xác suất (Likelihood Displacement) & RPO [45, 51]

Học viên thường trực giác rằng khi huấn luyện DPO, xác suất của câu tốt $\log \pi_\theta(y_w \mid x)$ sẽ tăng lên và xác suất của câu dở $\log \pi_\theta(y_l \mid x)$ sẽ giảm xuống. Tuy nhiên, công trình của Razin et al. [51] tại ICLR 2025 và Pal et al. [44] đã phát hiện một hiện tượng gây chấn động: **Trong quá trình huấn luyện DPO, xác suất tuyệt đối của câu trả lời tốt** $\log \pi_\theta(y_w \mid x)$ **thực chất cũng bị sụt giảm!**



Hình 3.1: Hiện tượng Dịch chuyển Xác suất (Likelihood Displacement): Cả y_w và y_l đều giảm xác suất, nhưng vì y_l giảm nhanh hơn nên mất mát DPO vẫn giảm.

* **Cơ chế toán học của Likelihood Displacement:** Trong các tác vụ giải toán hoặc lập trình, câu trả lời đúng y_w và câu trả lời sai y_l thường có tới $80\% - 90\%$ các bước dẫn xuất

đầu tiên giống hệt nhau, chỉ khác nhau ở một con số ở phép tính cuối cùng. Khi lực đạo hàm $-\nabla_\theta \log \pi_\theta(y_l \mid x)$ đè mạnh các token của y_l, nó vô tình đè bẹp luôn các token chung nằm trong y_w! Chỉ cần $\log \pi_\theta(y_l)$ tụt nhanh hơn $\log \pi_\theta(y_w)$ (ví dụ y_l tụt từ -10 xuống -50, còn y_w tụt từ -10 xuống -25), thì hiệu số $(-25) - (-50) = +25$ vẫn tăng mạnh và hàm mất mát DPO vẫn giảm đẹp đẽ! * **Hậu quả:** Do tổng xác suất trên toàn bảng từ vựng phải bằng 1 ($\sum_y \pi_\theta(y \mid x) = 1$), khi cả y_w và y_l đều bị rút bớt khối lượng xác suất, phần xác suất bị mất đó sẽ **tràn sang các chuỗi token kỳ quặc nằm ngoài phân phối (Out-of-Distribution)**, làm suy giảm nghiêm trọng khả năng lập luận của mô hình. * **Giải pháp chuẩn công nghiệp: DPO+NLL / RPO [17, 35, 45]:** Cộng thêm một thành phần hàm mất mát SFT (Negative Log-Likelihood) tính riêng trên câu trả lời tốt y_w với trọng số $\alpha > 0$:

$$\boxed{\mathcal{L}_{\text{DPO+NLL}}(\theta) = \mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) + \alpha \cdot \left(-\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log \pi_\theta(y_{w,t} \mid x, y_{w,<t}) \right)} \tag{3.21}$$

Thành phần SFT đóng vai trò như một chiếc mỏ neo kéo xác suất của y_w không bị trôi tuột xuống.

3.3.3 Cạm bẫy 3: Phân rã Đơn nhất không cần Reference Model (ORPO Hong et al. 27)

Phương pháp **ORPO (Odds Ratio Preference Optimization)** [27] đề xuất gộp chung giai đoạn SFT và giai đoạn Alignment vào làm một bước duy nhất bằng cách sử dụng tỷ số Odds:

$$\text{Odds}_\theta(y \mid x) = \frac{\pi_\theta(y \mid x)}{1 - \pi_\theta(y \mid x)}. \tag{3.22}$$

Hàm mất mát ORPO kết hợp hàm mất mát SFT trên y_w và phạt tỷ số Odds giữa y_w và y_l:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{SFT}}(y_w) + \lambda_{\text{OR}} \cdot \left(-\log \sigma \left(\log \frac{\text{Odds}_\theta(y_w \mid x)}{\text{Odds}_\theta(y_l \mid x)} \right) \right). \tag{3.23}$$

3.3.4 Cạm bẫy 4: Suy biến khi Sở thích Tất định (IPO) & Tối ưu theo Lý thuyết Triển vọng (KTO)

- **IPO (Identity Preference Optimization – Azar et al., Google DeepMind, AISTATS 2024 Azar et al. 4):** Chỉ ra rằng trong mô hình Bradley–Terry, nếu dữ liệu sở thích là tất định ($P(y_w \succ y_l) = 1$), DPO sẽ cố đẩy hiệu số phần thưởng ra vô cùng $+\infty$ bất kể β lớn đến đâu, dẫn đến việc mô hình luôn bị trôi dạt khỏi π_{ref}. IPO khắc phục bằng cách thay hàm log-sigmoid bằng một hàm mất mát bình phương khoảng cách:

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)} \left[\left(\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \frac{1}{2\beta} \right)^2 \right]. \tag{3.24}$$

- **KTO (Kahneman–Tversky Optimization – Ethayarajh et al., ICML 2024**

Ethayarajh et al. 19): Dựa trên *Lý thuyết Triển vọng (Prospect Theory)* đạt giải Nobel Kinh tế của Kahneman và Tversky (con người ghét sự mất mát hơn là yêu thích phần thưởng cùng giá trị). KTO cho phép huấn luyện mô hình căn chỉnh trực tiếp từ **dữ liệu phản hồi đơn lẻ** (mỗi câu trả lời chỉ cần gắn nhãn nhị phân: Thích hoặc Không thích, ví dụ nút Thumbs Up/Down trong sản phẩm thực tế) mà không cần phải tốn công ghép thành cặp (y_w, y_l) cho cùng một câu hỏi.

3.3.5 Offline DPO vs. Online / Iterative DPO [69]

Trong DPO tiêu chuẩn (**Offline DPO**), tập dữ liệu (x, y_w, y_l) được tạo sẵn cố định từ trước. Khi mô hình π_θ thay đổi qua hàng nghìn bước cập nhật, phân phối của π_θ đã trôi xa khỏi phân phối sinh ra tập dữ liệu tĩnh ban đầu (**Distribution Shift / Off-Policy Gap**). Để khắc phục, các hệ thống tiên tiến nhất như Llama 3 [17] áp dụng **Iterative / Online DPO**: cứ sau mỗi chu kỳ huấn luyện, lại dùng chính phiên bản π_θ mới nhất sinh ra các cặp câu trả lời mới, chấm điểm qua Reward Model hoặc bộ kiểm chứng, rồi tiếp tục huấn luyện vòng DPO tiếp theo.

Chương 4

Kỷ nguyên Reasoning RL: RLVR, Giải thuật GRPO, DeepSeek-R1 & Test-Time Compute

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 4.1. Chương này đưa người học bước vào đường biên nghiên cứu hiện đại nhất của trí tuệ nhân tạo (giai đoạn 2024–2026): **Kỷ nguyên Reasoning RL**. Chúng ta sẽ giải thích bản chất nhận thức của Hệ thống 2 (*System 2 Thinking*), phân tích sự dịch chuyển từ RLHF sang RLVR, giải phẫu cuộc tranh luận lịch sử giữa PRM và ORM, phân tích hiện tượng chuỗi suy luận tự phát trong **DeepSeek-R1-Zero** và quy trình 4 bước của **DeepSeek-R1**, dẫn xuất toán học toàn diện của giải thuật **GRPO**, chứng minh ba bộ ước lượng KL của Schulman, vạch rõ hai thiên kiến toán học trong GRPO gốc dẫn đến **Dr. GRPO / DAPO**, và khám phá quy luật mở rộng tính toán lúc suy luận (*Test-Time Compute Scaling*).

4.1 Sự Dịch chuyển Mô thức: Từ RLHF sang RLVR (Reinforcement Learning with Verifiable Rewards)

Trong tâm lý học nhận thức [32], bộ não con người xử lý thông tin thông qua hai cơ chế tư duy:

- **Hệ thống 1 (System 1 – Tư duy Nhanh, Bản năng):** Phản xạ tức thì, trực giác, không cần tốn năng lượng tính toán. Đây chính là cách một LLM thông thường hoạt động khi dự đoán token tiếp theo: với một câu hỏi hóc búa, việc ép mô hình phải xuất ngay đáp số ở token đầu tiên là trái với quy luật tự nhiên, vì số lượng phép tính ma trận trong một lượt đi qua mạng Transformer là cố định!
- **Hệ thống 2 (System 2 – Tư duy Chậm, Logic, Có Chủ đích):** Con người dùng giấy nháp để viết ra từng bước trung gian, thử nghiệm các giả thuyết, phát hiện mâu thuẫn, quay lui (*backtracking*) và kiểm tra lại trước khi chốt kết luận cuối cùng. Trong LLM, đây chính là **Chuỗi Suy luận (Chain-of-Thought – CoT)** [67].

Trước năm 2024, Post-training chủ yếu tập trung vào RLHF để uốn nắn văn phong trò chuyện lịch sự (System 1). Để kích hoạt năng lực suy luận System 2 vượt bậc, phương thức **RLVR (Reinforcement Learning with Verifiable Rewards)** [15, 35] ra đời, thay đổi tận gốc nguồn tín hiệu phần thưởng (Bảng 4.1).

Tiêu chí So sánh	RLHF Truyền thống (Chat Alignment)	RLVR Hiện đại (Reasoning Post-Training)
Nguồn Tín hiệu Thưởng	**Mạng nơ-ron Reward Model r_ϕ:** Học xấp xỉ cảm nhận chủ quan của con người (dễ bị nhiễu và thiên vị).	**Trình kiểm chứng Tất định (Deterministic Verifier):** Hàm kiểm tra khách quan bằng quy tắc logic (Regex kết quả toán, Compiler + Unit Tests cho code).
Nguy cơ Hack Thưởng (Reward Hacking)	**Rất cao:** Mô hình dễ dàng lừa mạng nơ-ron r_ϕ bằng cách nói dài dòng, nịnh bợ hoặc dùng từ ngữ hoa mỹ rỗng tuếch.	**Bằng 0 đối với kết quả:** Không thể "nịnh bợ" trình biên dịch Python hay một đẳng thức toán học; kết quả chỉ có Đúng (+1) hoặc Sai (0).
Quy mô Số bước Chạy RL	**Ngắn:** Thường dừng sớm sau vài trăm bước trước khi Reward Model bị phá vỡ (*Early Stopping*).	**Cực dài:** Có thể chạy liên tục hàng chục nghìn bước RL mà chất lượng suy luận vẫn tăng đều đặn.
Hành vi Hình thành	Văn phong trau chuốt, an toàn, biết từ chối các câu hỏi nhạy cảm.	Tự phát sinh chuỗi suy luận dài ($\langle$think$\rangle \dots$), tự thử sai, quay lui và tự kiểm chứng.

Bảng 4.1: Đối chiếu toàn diện giữa RLHF truyền thống và RLVR hiện đại.

4.2 Cuộc Tranh Luận ORM vs. PRM & Hiện tượng Tự phát trong DeepSeek-R1-Zero

Khi huấn luyện mô hình giải một bài toán suy luận gồm 20 bước, có hai trường phái thiết kế phần thưởng:

4.2.1 1. PRM (Process Reward Model – Lightman et al., OpenAI, 2023 Lightman et al. 37)

- **Ý tưởng:** Một mạng nơ-ron giám khảo PRM sẽ chấm điểm Đúng/Sai cho **từng bước suy luận trung gian** (giám sát dày đặc – *Dense Supervision*) [37, 63].
- **Rào cản chí mạng khi mở rộng quy mô [15]:**
  1. Rất khó định nghĩa ranh giới rõ ràng của một "bước suy luận" trong tư duy tự do, đặc biệt khi mô hình cố tình đi vào một nhánh sai để khám phá rồi tự quay lui.
  2. Chi phí thuê chuyên gia gán nhãn hàng triệu bước toán học là quá đắt đỏ.
  3. Nếu huấn luyện một mạng nơ-ron làm PRM tự động, chính sách π_θ sẽ lập tức **Hack PRM**: viết ra hàng trăm bước biến đổi hiển nhiên (như $x = x, 1 = 1$) để gom điểm thưởng dương vô tận!

4.2.2 2. ORM / Rule-based Outcome Verifier & Thí nghiệm Lịch sử của DeepSeek-R1-Zero [15]

Nhóm nghiên cứu DeepSeek đã đưa ra một quyết định táo bạo: **Bỏ qua hoàn toàn giai đoạn SFT ban đầu** (không dùng bất kỳ bài giải mẫu nào của con người) và **bỏ qua hoàn toàn PRM**! Họ lấy thẳng Base Model (*DeepSeek-V3-Base*) và chạy Học tăng cường trực tiếp chỉ với **hai phần thưởng luật khách quan (Rule-based Rewards)**:

1. **Phần thưởng Độ chính xác (Accuracy Reward $r_{\mathbf{acc}}$):** Kiểm tra xem kết quả cuối cùng có khớp chính xác tuyệt đối với đáp án toán học hoặc vượt qua 100% các bài kiểm thử mã nguồn (*Unit Tests*) hay không: $r_{\text{acc}} \in \{0, 1\}$.
2. **Phần thưởng Định dạng (Format Reward $r_{\mathbf{format}}$):** Kiểm tra xem mô hình có đặt toàn bộ quá trình suy nghĩ vào trong cặp thẻ $\langle$think$\rangle \dots \langle$/think$\rangle$ và đặt kết quả cuối cùng vào trong thẻ $\langle$answer$\rangle \dots \langle$/answer$\rangle$ hay không.

4.2.3 Sự Trỗi dậy Tự nhiên của Chuỗi Suy luận Dài & Khoảnh khắc "Aha"

Kết quả của thí nghiệm DeepSeek-R1-Zero đã làm thay đổi hoàn toàn nhận thức của cộng đồng AI thế giới:

- Dù không có bất kỳ dòng dữ liệu SFT nào dạy mô hình phải tư duy ra sao trong thẻ $\langle$think$\rangle$, qua hàng nghìn bước cập nhật RL, **độ dài trung bình của chuỗi suy nghĩ tự động tăng vọt từ vài trăm token lên tới hàng nghìn token**!
- **Cơ chế toán học đằng sau hiện tượng tự phát này:** Trong quá trình khám phá ngẫu nhiên của RL, với cùng một bài toán khó, những quỹ đạo nào trả lời vội vàng đều dẫn đến đáp số sai ($r = 0$). Ngược lại, những quỹ đạo nào tình cờ dùng các token trung gian để phân tích kỹ đề bài, thử nghiệm các hướng đi khác nhau, hoặc khi thấy mâu thuẫn biết dừng lại tính lại từ đầu sẽ có **xác suất chạm tới đáp số đúng ở cuối câu cao hơn hẳn** ($r = +1$). Đạo hàm Policy Gradient củng cố mạnh mẽ các quỹ đạo đúng. Mô hình tự rút ra chiến lược tối ưu: **Dùng các token bên trong thẻ $\langle$think$\rangle$ làm không gian tính toán nháp để tăng độ sâu suy luận!**
- **Khoảnh khắc "Aha" (Aha Moment):** Ở giữa quá trình huấn luyện, mô hình tự phát sinh hành vi nhận thức phản tỉnh: khi đang giải dở một phương trình phức tạp, nó tự viết: *"Wait, wait. Wait. That's an aha moment I can flag here... Let me re-evaluate this step from the beginning..."*

4.2.4 Từ R1-Zero đến Quy trình 4 Giai đoạn Chuẩn mực của DeepSeek-R1

Mặc dù DeepSeek-R1-Zero giải toán cực giỏi (tăng điểm AIME 2024 từ 15.6% lên 71.0%), nhưng vì không có SFT làm khuôn mẫu, nó mắc lỗi **Trộn lẫn Ngôn ngữ (Language Mixing)** (đang hỏi tiếng Anh lại suy nghĩ bằng tiếng Trung vì RL chỉ quan tâm đáp số đúng). Để tạo ra mô hình **DeepSeek-R1** thương mại hoàn chỉnh, quy trình Post-training chuẩn hiện đại gồm **4 giai đoạn nối tiếp**:

1. **Giai đoạn 1 – Khởi động Lạnh (Cold-Start SFT):** Tinh chỉnh Base Model trên một tập nhỏ (vài nghìn mẫu) dữ liệu Long-CoT sạch đẹp, có cấu trúc rõ ràng để định hình thẩm mỹ trình bày.

2. **Giai đoạn 2 – RL Định hướng Suy luận (Reasoning RL / RLVR):** Chạy thuật toán GRPO quy mô lớn trên các bài toán Toán, Code và Logic có bổ sung phần thưởng phạt tính bất nhất quán ngôn ngữ.

3. **Giai đoạn 3 – Lấy mẫu Bác bỏ & SFT Toàn diện (Rejection Sampling + General SFT):** Dùng mô hình hội tụ ở Giai đoạn 2 sinh ra khoảng $600,000$ mẫu suy luận đúng, gộp với $200,000$ mẫu SFT tổng quát (viết lách, dịch thuật, trò chuyện an toàn) để huấn luyện một mô hình SFT mới hoàn chỉnh.

4. **Giai đoạn 4 – RL Đa Tình huống (All-Scenario RL):** Vòng RL cuối cùng kết hợp cả Verifiable Rewards (cho toán/code) và Reward Model (cho tính hữu ích và an toàn xã hội).

4.3 Giải thuật GRPO (Group Relative Policy Optimization – Shao et al., 2024)

Thuật toán RL làm nên xương sống của DeepSeekMath [56] và DeepSeek-R1 [15] là **GRPO (Group Relative Policy Optimization)**.

4.3.1 Tại sao PPO Thất bại với Reasoning Chuỗi Dài?

Trong PPO (Chương 2), mạng Critic V_ψ có kích thước ngang ngửa Policy π_θ. Với các chuỗi suy luận dài từ $8,000$ đến $32,000$ tokens, việc huấn luyện Critic dự đoán chính xác triển vọng điểm số tại từng token là cực kỳ bất ổn định, đồng thời mạng Critic ngốn thêm một lượng VRAM khổng lồ tương đương với Actor.

4.3.2 Ý tưởng Cốt lõi của GRPO: Xóa sổ Hoàn toàn Mạng Critic!

Shao et al. [56] nhận ra: *"Mục đích thực sự của Critic $V_\psi(q)$ trong PPO chỉ là cung cấp một giá trị kỳ vọng cơ sở (Baseline) để biết câu trả lời hiện tại tốt hơn hay tệ hơn mức trung bình của câu hỏi q. Tại sao ta phải tốn hàng chục Gigabyte VRAM để huấn luyện một mạng nơ-ron đoán mò giá trị đó, trong khi ta có thể trực tiếp cho mô hình sinh ra một nhóm G câu trả lời cho cùng câu hỏi q rồi lấy điểm trung bình thực nghiệm của nhóm đó?"*

4.3.3 Bốn Bước Toán học của Giải thuật GRPO

1. **Bước 1 – Lấy mẫu theo Nhóm (Group Sampling):** Với mỗi câu hỏi đầu vào $q \sim \mathcal{D}$, dùng chính sách hiện tại $\pi_{\theta_{\text{old}}}$ sinh ra một nhóm gồm G câu trả lời độc lập (thường chọn $G \in [8, 64]$):

$$\{o_1, o_2, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q). \tag{4.1}$$

2. **Bước 2 – Đánh giá qua Bộ kiểm chứng (Verification):** Đưa cả G câu trả lời qua bộ kiểm chứng để nhận về G điểm thưởng tương ứng: $\{r_1, r_2, \ldots, r_G\}$.

3. **Bước 3 – Tính Lợi thế Tương đối theo Nhóm (Group Relative Advantage):** Tính giá trị trung bình mean(r) và độ lệch chuẩn std(r) trong nhóm:

$$\bar{r} = \frac{1}{G}\sum_{i=1}^{G} r_i, \quad \sigma_r = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i - \bar{r})^2}. \tag{4.2}$$

Chuẩn hóa điểm thưởng của câu trả lời thứ i thành giá trị lợi thế vô hướng A_i:

$$\boxed{A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}} \tag{4.3}$$

(với $\epsilon \approx 10^{-8}$ là hằng số chống chia cho 0).

4. **Bước 4 – Tối ưu hóa Hàm Mục tiêu Cắt ngưỡng của GRPO:**

$$\boxed{\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \left(\min\left(\rho_{i,t}(\theta)A_i,\ \text{clip}\left(\rho_{i,t}(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}\right)A_i \right) - \beta\,\mathbb{D}_{\text{KL}} \right.} \tag{4.4}$$

trong đó tỷ số xác suất tại token thứ t của câu trả lời thứ i là:

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}. \tag{4.5}$$

4.3.4 Mở rộng GRPO cho Giám sát Quá trình (Process Supervision)

Khi áp dụng cho PRM (nơi mỗi bước suy luận j nhận một điểm thưởng $r_{i,j}$ riêng), công thức lợi thế tại bước j được chuẩn hóa tương đối theo nhóm các bước tương ứng:

$$A_{i,j} = \frac{r_{i,j} - \text{mean}\left(\{r_{k,j}\}_{k=1}^{G}\right)}{\text{std}\left(\{r_{k,j}\}_{k=1}^{G}\right) + \epsilon}, \tag{4.6}$$

và mọi token t thuộc phạm vi bước j sẽ nhận chung giá trị lợi thế $A_{i,t} = A_{i,j}$.

4.4 Chứng minh Ba Bộ Ước lượng KL của Schulman (k_1, k_2, k_3)

Trong công thức GRPO ở Phương trình (4.4), thành phần phạt phân kỳ KL tại từng token $\mathbb{D}_{\text{KL}}\left(\pi_\theta \parallel \pi_{\text{ref}}\right)_{i,t}$ được tính như thế nào? Trong bài viết kinh điển của mình, Schulman [53] đã phân tích ba bộ ước lượng điểm mẫu (*Sample-based Estimators*) cho $\mathbb{D}_{\text{KL}}(P \parallel Q)$ khi lấy mẫu từ $x \sim P$:

Đặt tỷ số khả dĩ $r(x) = \frac{Q(x)}{P(x)} > 0$. Ta có:

Định lý 4.1 (Ba Bộ Ước lượng Phân kỳ KL của Schulman [53]). *(a)* ***Bộ ước lượng Thô*** *(k_1)**:***

$$k_1 = \log \frac{P(x)}{Q(x)} = -\log r(x). \tag{4.7}$$

(b) ***Bộ ước lượng Bậc hai*** *(k_2)**:***

$$k_2 = \frac{1}{2}\left(\log \frac{P(x)}{Q(x)}\right)^2 = \frac{1}{2}(\log r(x))^2. \tag{4.8}$$

(c) ***Bộ ước lượng Không chệch Luôn Dương*** *(k_3* ***– Bộ ước lượng dùng trong GRPO)**:***

$$k_3 = \frac{Q(x)}{P(x)} - \log \frac{Q(x)}{P(x)} - 1 = r(x) - \log r(x) - 1. \tag{4.9}$$

Chứng minh Tính chất của Bộ ước lượng k_3. **1. Chứng minh tính không âm tuyệt đối tại mọi mẫu ($k_3 \geq 0, \forall x$):** Xét hàm số thực $f(r) = r - \log r - 1$ xác định trên $(0, +\infty)$. Đạo hàm bậc nhất: $f'(r) = 1 - \frac{1}{r} = \frac{r-1}{r}$. Ta có: $f'(r) = 0 \iff r = 1$. Khi $r < 1$, $f'(r) < 0$ (hàm nghịch biến); khi $r > 1$, $f'(r) > 0$ (hàm đồng biến). Do đó, $f(r)$ đạt giá trị cực tiểu toàn cục duy nhất tại $r = 1$:

$$f_{\min} = f(1) = 1 - \log(1) - 1 = 0 \implies k_3 = r - \log r - 1 \geq 0, \quad \forall r > 0. \tag{4.10}$$

Đẳng thức xảy ra khi và chỉ khi $r = 1$, tức $P(x) = Q(x)$.

2. Chứng minh tính không chệch ($\mathbb{E}_{x\sim P}[k_3] = \mathbb{D}_{\mathrm{KL}}(P \parallel Q)$): Lấy kỳ vọng của k_3 theo phân phối $x \sim P$:

$$\begin{aligned}
\mathbb{E}_{x\sim P}[k_3] &= \mathbb{E}_{x\sim P}\left[\frac{Q(x)}{P(x)} - \log \frac{Q(x)}{P(x)} - 1\right] \\
&= \sum_x P(x)\frac{Q(x)}{P(x)} + \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right] - \mathbb{E}_{x\sim P}[1] \\
&= \underbrace{\sum_x Q(x)}_{=1} + \mathbb{D}_{\mathrm{KL}}(P \parallel Q) - 1 = \mathbb{D}_{\mathrm{KL}}(P \parallel Q).
\end{aligned} \tag{4.11}$$

□

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 4.2 (Tại sao GRPO bắt buộc phải dùng k_3 thay vì k_1? Nếu dùng bộ ước lượng ngây thơ $k_1 = -\log r$: Khi một mẫu ngẫu nhiên rơi vào vùng mà $\pi_{\mathrm{ref}}(y_t) > \pi_\theta(y_t)$, ta có $r > 1 \implies k_1 = -\log r < 0$! Việc một giá trị KL nhận giá trị **âm** tại từng token đơn lẻ sẽ biến mức phạt thành phần thưởng giả tạo, làm bùng nổ phương sai của gradient.

- Bộ ước lượng k_3 bảo đảm **mọi token ở mọi bước luôn nhận một mức phạt không âm** ≥ 0, đồng thời kỳ vọng toán học chuẩn xác 100% không bị lệch.

4.5 Hai Thiên kiến Toán học trong GRPO Gốc, Dr. GRPO & DAPO

Mặc dù GRPO rất thành công, công trình giải phẫu toán học gần đây của Liu et al. [38] (**Dr. GRPO**) và Yu et al. [70] (**DAPO**) đã chỉ ra hai thiên kiến tinh vi trong công thức GRPO gốc:

4.5.1 1. Thiên kiến Độ dài Câu trả lời (Response-level Length Bias) do Hệ số $\frac{1}{|o_i|}$

Hãy nhìn vào hệ số lấy trung bình theo chiều dài câu trả lời $\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}$ trong Phương trình (4.4):

- **Khi câu trả lời thứ i Đúng ($A_i > 0$):** Mục tiêu là tối đa hóa đại lượng dương $\frac{A_i}{|o_i|}$. Để giá trị này lớn nhất, mô hình nhận thấy câu trả lời càng ngắn ($|o_i|$ nhỏ) thì giá trị trung bình trên mỗi token càng cao $\implies$ Khuyến khích câu trả lời đúng ngắn gọn.
- **Khi câu trả lời thứ i Sai ($A_i < 0$):** Mục tiêu là làm cho mức phạt âm $\frac{A_i}{|o_i|}$ bớt âm nhất (tiến về 0). Mô hình phát hiện một kẽ hở toán học: **Nếu nó cố tình kéo dài câu trả lời sai ra vô tận ($|o_i| \to \infty$), mức phạt âm trên mỗi token sẽ bị pha loãng về 0!**

Đây chính là nguyên nhân giải thích tại sao trong quá trình huấn luyện DeepSeek-R1-Zero, các câu trả lời sai của mô hình ngày càng dài dòng và lặp từ bất tận.

4.5.2 2. Thiên kiến Độ khó Câu hỏi (Question-level Difficulty Bias) do Phép chia $\text{std}(r)$

Khi tính $A_i = \frac{r_i - \bar{r}}{\text{std}(r)+\epsilon}$, những câu hỏi cực dễ (15/16 mẫu đúng) hoặc cực khó (15/16 mẫu sai) sẽ có phương sai điểm thưởng cực nhỏ ($\text{std}(r) \to 0$). Việc chia cho một số cực nhỏ sẽ **phóng đại quá mức lợi thế A_i của các câu hỏi dễ/khó vô lý**, làm gradient bị chi phối bởi các câu hỏi ngoại lai thay vì các câu hỏi vừa sức.

4.5.3 Giải pháp của Dr. GRPO [38] & DAPO [70]

- **Dr. GRPO (Liu et al., 2025):** Thay thế hệ số biến thiên $\frac{1}{|o_i|}$ bằng một hằng số chuẩn hóa cố định $L_{\max}$ trên toàn bộ tập dữ liệu, và loại bỏ phép chia cho $\text{std}(r)$ trong công thức Advantage:

$$A_i^{\text{Dr.GRPO}} = r_i - \bar{r}, \quad \text{và dùng trọng số } \frac{1}{L_{\max}}\sum_{t=1}^{|o_i|}. \tag{4.12}$$

- **DAPO (Yu et al., ByteDance & Tsinghua, 2025 Yu et al. 70):** Đưa ra cơ chế **Lọc Động (Dynamic Sampling Policy Optimization)**: tự động loại bỏ các câu hỏi có toàn bộ nhóm đúng hết ($r_1 = \cdots = r_G = 1$) hoặc sai hết ($r_1 = \cdots = r_G = 0$) trong lúc tạo batch, chỉ giữ lại các câu hỏi có $0 < \bar{r} < 1$ để đảm bảo gradient luôn chứa thông tin đối chiếu có ý nghĩa.

4.6 Quy luật Mở rộng Tính toán lúc Suy luận (Test-Time Compute Scaling)

Sự thành công của Reasoning RL đã khai sinh một quy luật mở rộng mới bên cạnh Định luật Chinchilla [26]: **Quy luật Mở rộng Tính toán lúc Suy luận (Test-Time Compute Scaling)** [8, 58].

4.6.1 Mở rộng Song song vs. Mở rộng Tuần tự

1. **Mở rộng Song song (Parallel Scaling – Best-of-N / Majority Voting Wang et al. 65):** Cho mô hình sinh ra N câu trả lời độc lập cùng lúc và lấy kết quả theo số đông (*Self-Consistency*).
2. **Mở rộng Tuần tự (Sequential Scaling – Long CoT):** Cho mô hình sinh một chuỗi suy luận dài nối tiếp nhau trong thẻ ⟨think⟩.

Định lý 4.2 (Sự Ưu việt của Mở rộng Tuần tự so với Mở rộng Song song [58])**.** *Đối với các bài toán suy luận phức tạp, việc cấp thêm ngân sách tính toán (FLOPs) cho chuỗi suy luận tuần tự qua RL mang lại hiệu suất vượt trội hơn việc lấy mẫu song song Best-of-N.*

Trực giác Toán học. Trong N nhánh chạy song song, các nhánh hoàn toàn độc lập và không thể giao tiếp với nhau; nếu một định kiến sai lầm có xác suất cao trong Base Model, cả N nhánh đều có xu hướng cùng vấp phải lỗi sai đó. Ngược lại, trong một chuỗi suy luận tuần tự dài, cơ chế **Causal Self-Attention** cho phép các token ở bước thứ 2,000 nhìn thấy toàn bộ các bước thử nghiệm thất bại ở bước thứ 500, từ đó rút kinh nghiệm, loại trừ nhánh cụt và điều chỉnh hướng đi để tìm ra lời giải chính xác. □

4.6.2 Công thức Ước lượng Không chệch Pass@k của Chen et al. (2021)

Khi đánh giá năng lực giải toán và viết mã nguồn với ngân sách k lần thử, ta cho mô hình sinh $n \geq k$ mẫu (ví dụ $n = 100, k = 1$). Gọi c là số mẫu giải đúng trong n mẫu ($c \leq n$).

Định nghĩa 4.1 (Bộ Ước lượng Không chệch Pass@k [10])**.** Thay vì lấy mẫu ngẫu nhiên lặp lại nhiều lần gây phương sai, giá trị kỳ vọng không chệch của Pass@k được tính chính xác bằng giải tích tổ hợp:

$$\boxed{\text{Pass@}k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]} \tag{4.13}$$

Ý nghĩa: $\frac{\binom{n-c}{k}}{\binom{n}{k}}$ là xác suất để cả k mẫu được chọn ngẫu nhiên đều rơi vào tập $n - c$ mẫu bị sai. Trừ đi xác suất này cho ta xác suất có ít nhất một mẫu đúng.

Chương 5

Chắt lọc Tri thức Mô hình (Model Distillation) trong Post-Training

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 5.1. Chương này hoàn thiện bức tranh về ba trụ cột của Post-training: **Tối ưu hóa Sở thích (Preference Alignment)**, **Học Tăng Cường Suy luận (Reasoning RL)**, và **Chắt lọc Tri thức (Model Distillation)**. Chúng ta sẽ giải thích bản chất toán học của mô hình Thầy–Trò (*Teacher–Student Paradigm*), phân tích sự đối đầu kinh điển giữa Phân kỳ KL Xuôi (*Forward KL*) và Phân kỳ KL Ngược (*Reverse KL*) trong sinh văn bản tự hồi quy, tìm hiểu kỹ thuật chắt lọc trên chính sai lầm của học sinh (*On-Policy Distillation*), và khám phá ba hình thái chuyển giao tri thức: phân phối xác suất (*Logits*), chuỗi đầu ra (*Sequence*) và lý giải suy luận (*Rationale*).

5.1 Mô hình Thầy–Trò trong Post-Training: Động lực & Khái niệm

Trong khi Pre-training đòi hỏi chi phí tính toán khổng lồ để hấp thụ toàn bộ kho văn bản thô của nhân loại, thì ở giai đoạn Post-training, một trong những mục tiêu thực tiễn quan trọng nhất là: **Làm thế nào để chuyển giao năng lực tư duy và hành vi chuẩn mực từ một mô hình khổng lồ (Teacher Model π_{Teacher}, ví dụ 671B tham số) sang một mô hình học sinh nhỏ gọn (Student Model π_{Student}, ví dụ 7B hoặc 8B tham số) để có thể triển khai trên các thiết bị biên?**

Quy trình này gọi là **Chắt lọc Tri thức (Knowledge Distillation – KD)** [25]. Khác với việc huấn luyện mô hình nhỏ từ đầu trên dữ liệu thô, trong Post-training, mô hình học sinh được tiếp cận trực tiếp với các tín hiệu giám sát giàu thông tin ngữ nghĩa từ mô hình giáo viên (phân phối xác suất mềm, chuỗi suy luận mẫu và khả năng tự phản biện).

5.2 Hình thái 1: Chắt lọc Phân phối Xác suất (Logit Distillation) & Nhiệt độ τ

Hình thái kinh điển nhất của chắt lọc tri thức xuất phát từ công trình của Hinton et al. [25]. Tại mỗi bước thời gian t, xét vector trạng thái ẩn của học sinh $\mathbf{h}_t^{\mathrm{S}}$ và của giáo viên $\mathbf{h}_t^{\mathrm{T}}$. Lớp chiếu đầu ra sinh ra vector logits tương ứng $\mathbf{z}_t^{\mathrm{S}} \in \mathbb{R}^{|V|}$ và $\mathbf{z}_t^{\mathrm{T}} \in \mathbb{R}^{|V|}$.

5.2.1 Hàm Softmax với Tham số Nhiệt độ $\tau > 0$

Trong phân loại thông thường ($\tau = 1$), hàm Softmax $P(i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$ có xu hướng dồn gần như 100% khối lượng xác suất vào token có logit lớn nhất ($P(y^*) \approx 0.99$), trong khi hàng nghìn token còn lại nhận xác suất xấp xỉ 0 (10^{-5} đến 10^{-8}). Tuy nhiên, trong các xác suất cực nhỏ đó ẩn chứa một kho tàng thông tin ngữ nghĩa mà Hinton et al. [25] gọi là **Tri thức Ẩn (Dark Knowledge)**.

Ví dụ Minh họa 5.1 (Tri thức Ẩn trong Phân phối Xác suất)**.** Xét câu tiếng Anh: *"The animal didn't cross the street because it was too ..."* Token tiếp theo có xác suất cao nhất là `tired`. Nhưng trong số các token sai, giáo viên sẽ gán xác suất cho từ `scared` cao hơn gấp 10,000 lần so với từ `refrigerator`! Tỷ số tương đối giữa các token không được chọn phản ánh cấu trúc hình học ngữ nghĩa sâu sắc của không gian ngôn ngữ mà một nhãn cứng nhị phân (*One-hot Label*) không bao giờ truyền tải được.

Để khai mở tri thức ẩn này, ta đưa vào tham số **Nhiệt độ (Temperature $\tau > 0$)**:

$$P_\tau(x_t = i \mid x_{<t}) = \frac{\exp\left(z_i/\tau\right)}{\sum_{j=1}^{|V|} \exp\left(z_j/\tau\right)}. \tag{5.1}$$

Khi $\tau > 1$, phân phối xác suất được làm phẳng ra (*softened*), nâng các xác suất tiềm ẩn lên đủ lớn để tạo ra tín hiệu gradient hữu ích cho mô hình học sinh.

Định lý 5.1 (Xấp xỉ Dạo hàm khi Nhiệt độ $\tau \to \infty$ [25])**.** *Khi nhiệt độ τ đủ lớn so với độ lớn của logits ($|z_i| \ll \tau$), hàm mất mát phân kỳ KL giữa phân phối mềm của giáo viên P_τ^T và học sinh P_τ^S tương đương với hàm mất mát sai số bình phương trung bình (Mean Squared Error – MSE) trên các vector logits đã trừ trung bình:*

$$\lim_{\tau \to \infty} \nabla_{\mathbf{z}^S} \mathbb{D}_{\mathrm{KL}}\left(P_\tau^T \parallel P_\tau^S\right) \approx \frac{1}{|V|\tau^2}\left(\mathbf{z}^S - \mathbf{z}^T\right). \tag{5.2}$$

Chứng minh. Khai triển chuỗi Taylor cho hàm mũ quanh 0: $\exp(z_i/\tau) \approx 1 + \frac{z_i}{\tau}$. Mẫu số của hàm Softmax trở thành:

$$\sum_{j=1}^{|V|} \exp\left(\frac{z_j}{\tau}\right) \approx \sum_{j=1}^{|V|} \left(1 + \frac{z_j}{\tau}\right) = |V| + \frac{\sum_j z_j}{\tau} = |V|\left(1 + \frac{\bar{z}}{\tau}\right), \tag{5.3}$$

với $\bar{z} = \frac{1}{|V|}\sum_j z_j$. Xác suất mềm được xấp xỉ bằng:

$$P_\tau(i) \approx \frac{1 + z_i/\tau}{|V|(1 + \bar{z}/\tau)} \approx \frac{1}{|V|}\left(1 + \frac{z_i}{\tau}\right)\left(1 - \frac{\bar{z}}{\tau}\right) \approx \frac{1}{|V|}\left(1 + \frac{z_i - \bar{z}}{\tau}\right). \tag{5.4}$$

Đạo hàm của hàm mất mát Cross-Entropy mềm $\mathcal{L} = -\sum_i P_\tau^{\mathrm{T}}(i) \log P_\tau^{\mathrm{S}}(i)$ theo logit z_i^{S} là:

$$\frac{\partial \mathcal{L}}{\partial z_i^{\mathrm{S}}} = \frac{1}{\tau}\big(P_\tau^{\mathrm{S}}(i) - P_\tau^{\mathrm{T}}(i)\big) \approx \frac{1}{\tau}\left[\frac{1}{|V|}\left(1 + \frac{z_i^{\mathrm{S}} - \bar{z}^{\mathrm{S}}}{\tau}\right) - \frac{1}{|V|}\left(1 + \frac{z_i^{\mathrm{T}} - \bar{z}^{\mathrm{T}}}{\tau}\right)\right] = \frac{(z_i^{\mathrm{S}} - \bar{z}^{\mathrm{S}}) - (z_i^{\mathrm{T}} - \bar{z}^{\mathrm{T}})}{|V|\tau^2}. \tag{5.5}$$

Nếu các vector logits đã được chuẩn hóa về trung bình bằng 0 ($\bar{z} = 0$), đạo hàm tỷ lệ thuận trực tiếp với hiệu số sai lệch $z_i^{\mathrm{S}} - z_i^{\mathrm{T}}$. □

5.3 Cuộc Đối đầu Toán học: Forward KL (Mean-Seeking) vs. Reverse KL (Mode-Seeking)

Trong bài toán sinh văn bản tự hồi quy của LLM, việc chọn hướng phân kỳ KL nào để khớp phân phối giữa Thầy và Trò là một câu hỏi lý thuyết sâu sắc [1, 22]:

5.3.1 1. Phân kỳ KL Xuôi (Forward KL / Zero-Avoiding / Mean-Seeking)

Công thức chắt lọc truyền thống tối thiểu hóa phân kỳ KL từ Giáo viên sang Học sinh:

$$\mathcal{L}_{\mathrm{Forward}}(\theta) = \mathbb{D}_{\mathrm{KL}}\left(P_{\mathrm{Teacher}} \parallel P_{\mathrm{Student}}\right) = \sum_y P_{\mathrm{Teacher}}(y \mid x) \log \frac{P_{\mathrm{Teacher}}(y \mid x)}{P_{\mathrm{Student}}(y \mid x)}. \tag{5.6}$$

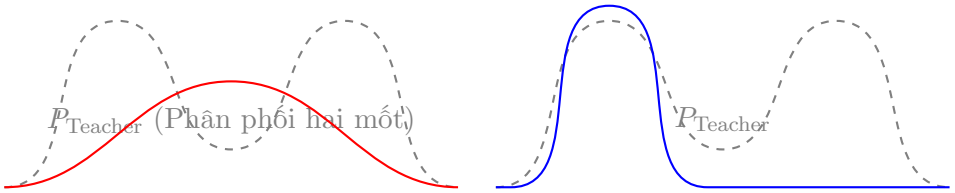
- **Đặc tính Toán học:** Bất cứ nơi nào giáo viên có xác suất $P_{\mathrm{Teacher}}(y) > 0$, nếu học sinh gán $P_{\mathrm{Student}}(y) \to 0$, thì tỷ số $\frac{P_{\mathrm{Teacher}}}{P_{\mathrm{Student}}} \to +\infty \implies \mathbb{D}_{\mathrm{KL}} \to +\infty$.
- **Hành vi thực tế:** Học sinh bị ép buộc phải **trải rộng khối lượng xác suất để bao phủ mọi vùng mà giáo viên có thể sinh ra** (*Zero-Avoiding / Mean-Seeking*).
- **Hậu quả trong LLM:** Dung lượng tham số của mô hình học sinh nhỏ hơn giáo viên rất nhiều (ví dụ 7B vs. 70B), nó không thể học được toàn bộ độ đa dạng của giáo viên. Việc ép nó bao phủ mọi mốt (*modes*) khiến nó phân bổ xác suất vào cả những vùng ranh giới trống rỗng giữa các mốt, dẫn đến hiện tượng **sinh câu lủng củng, ảo giác và mất tập trung** [22].

5.3.2 2. Phân kỳ KL Ngược (Reverse KL / Zero-Forcing / Mode-Seeking)

Ngược lại, nếu ta tối thiểu hóa phân kỳ KL từ Học sinh sang Giáo viên:

$$\mathcal{L}_{\mathrm{Reverse}}(\theta) = \mathbb{D}_{\mathrm{KL}}\left(P_{\mathrm{Student}} \parallel P_{\mathrm{Teacher}}\right) = \sum_y P_{\mathrm{Student}}(y \mid x) \log \frac{P_{\mathrm{Student}}(y \mid x)}{P_{\mathrm{Teacher}}(y \mid x)}. \tag{5.7}$$

- **Đặc tính Toán học:** Tại những vùng mà giáo viên không ủng hộ ($P_{\text{Teacher}}(y) \approx 0$), để tránh $\mathbb{D}_{\text{KL}} \to +\infty$, học sinh bắt buộc phải gán $P_{\text{Student}}(y) = 0$ (vì $\lim_{p \to 0} p \log p = 0$).
- **Hành vi thực tế:** Học sinh sẵn sàng bỏ qua nhiều mốt phức tạp của giáo viên để **tập trung toàn bộ khối lượng xác suất vào một vài mốt chất lượng cao nhất** (*Zero-Forcing / Mode-Seeking*).
- **Ưu điểm vượt trội cho LLM:** Trong sinh văn bản, người dùng chỉ cần mô hình trả lời chuẩn xác và mạch lạc theo một hướng đi đúng, chứ không cần mô hình phải nhớ hết mọi cách nói lan man. Do đó, Reverse KL tạo ra các mô hình học sinh có văn phong sắc sảo, tự tin và ít ảo giác hơn hẳn!



Forward KL: Trải rộng bao trùm cả vùng rỗng Reverse KL: Khóa chặt vào một mốt tối ưu

Hình 5.1: Minh họa hành vi toán học: Forward KL (Mean-Seeking) bao trùm cả vùng xác suất rỗng vs. Reverse KL (Mode-Seeking) tập trung chính xác vào mốt tốt nhất.

5.3.3 3. Chắt lọc Trực tuyến trên Chính Sai lầm của Học sinh (On-Policy Distillation – MiniLLM & GKD)

Tuy nhiên, có một rào cản tính toán lớn: Trong Phương trình (5.7), kỳ vọng được lấy theo phân phối của học sinh: $\mathbb{E}_{y \sim \pi_{\text{Student}}}$. Để tính đạo hàm, ta không thể dùng dữ liệu tĩnh mà phải cho học sinh tự sinh văn bản!

Công trình **MiniLLM** [22] và **GKD (Generalized Knowledge Distillation)** của Google DeepMind [1] giải quyết vấn đề này bằng cách thiết lập thuật toán **On-Policy Distillation**:

1. Với câu hỏi x, cho chính mô hình học sinh π_{Student} tự sinh ra câu trả lời $\hat{y} \sim \pi_{\text{Student}}(\cdot \mid x)$ (*On-Policy Generation*).
2. Đưa câu trả lời $\hat{y}$ do học sinh tự sinh vào mô hình giáo viên π_{Teacher} để giáo viên chấm điểm và cung cấp phân phối mục tiêu tại từng bước.
3. Cập nhật trọng số của học sinh bằng thuật toán Policy Gradient (giống RL) với hàm phần thưởng là log-likelihood của giáo viên:

$$r(x, \hat{y}) = \sum_{t=1}^{|\hat{y}|} \log \pi_{\text{Teacher}}(\hat{y}_t \mid x, \hat{y}_{<t}). \tag{5.8}$$

Nhờ được huấn luyện trực tiếp trên **những sai lầm do chính mình tự gây ra** (*Self-generated mistakes*), mô hình học sinh loại bỏ hoàn toàn hiện tượng Exposure Bias và đạt năng lực vượt trội so với chắt lọc tĩnh truyền thống [1].

5.4 Hình thái 2: Chắt lọc Cấp độ Chuỗi (Sequence-Level Distillation – Kim & Rush, 2016)

Khác với Logit Distillation (đòi hỏi phải lưu trữ và truyền tải tensor logits khổng lồ kích thước $B \times L \times |V|$ qua mạng), phương pháp **Chắt lọc Cấp độ Chuỗi (Sequence-Level Knowledge Distillation – SeqKD)** [34] hoạt động hoàn toàn trên không gian văn bản:

Định nghĩa 5.1 (Quy trình Sequence-Level Distillation)**.**

1. Cho mô hình giáo viên π_{Teacher} chạy giải mã tìm kiếm (*Greedy Search* hoặc *Beam Search*) trên tập câu hỏi $\{x^{(i)}\}$ để tạo ra các chuỗi phản hồi chất lượng cao nhất:

$$y^*_{\text{Teacher}}(x) = \arg\max_y \pi_{\text{Teacher}}(y \mid x). \tag{5.9}$$

2. Tạo lập tập dữ liệu chắt lọc tổng hợp: $\mathcal{D}_{\text{distill}} = \{(x^{(i)}, y^*_{\text{Teacher}}(x^{(i)}))\}$.

3. Huấn luyện mô hình học sinh π_{Student} trên $\mathcal{D}_{\text{distill}}$ bằng hàm mất mát SFT có giám sát thông thường với mặt nạ Prompt Masking (Chương 1):

$$\mathcal{L}_{\text{SeqKD}}(\theta) = -\,\mathbb{E}_{(x,y^*)\sim\mathcal{D}_{\text{distill}}} \left[\sum_{t=1}^{|y^*|} \log \pi_\theta(y^*_t \mid x, y^*_{<t}) \right]. \tag{5.10}$$

Đây chính là phương pháp chủ đạo được Meta sử dụng để tạo ra dữ liệu tổng hợp cho dòng mô hình Llama 3 [17], và được DeepSeek sử dụng để chắt lọc năng lực từ DeepSeek-R1 sang các mô hình nhỏ (Qwen-1.5B/7B/14B/32B và Llama-8B/70B) [15].

5.5 Hình thái 3: Chắt lọc Quá trình Suy luận (Rationale / Step-by-Step Distillation)

Khi chắt lọc các bài toán suy luận phức tạp, nếu giáo viên chỉ cung cấp câu hỏi x và đáp án cuối cùng y, mô hình học sinh nhỏ sẽ rất khó nắm bắt được bước nhảy logic.

Công trình **Distilling Step-by-Step!** của Đại học Washington và Google Research [28] đề xuất hình thái **Chắt lọc Quá trình Suy luận (Rationale Distillation)**:

- Thay vì chỉ sinh đáp án y, mô hình giáo viên được yêu cầu sinh ra **Chuỗi Lý giải Nháp (Rationale r)** đi kèm kết quả cuối cùng: $x \to (r, y)$.
- Mô hình học sinh được huấn luyện theo cơ chế **Học Đa Nhiệm vụ (Multi-task Learning)** bằng cách sử dụng hai tiền tố điều khiển:

$$\text{Nhiệm vụ 1 (Sinh Lý giải):} \quad \mathcal{L}_{\text{rationale}} = -\log \pi_\theta(r \mid x, \text{prefix} = \texttt{explain:}), \tag{5.11}$$

$$\text{Nhiệm vụ 2 (Dự đoán Đáp án):} \quad \mathcal{L}_{\text{label}} = -\log \pi_\theta(y \mid x, r, \text{prefix} = \texttt{predict:}). \tag{5.12}$$

Tổng hàm mất mát: $\mathcal{L}_{\text{Step-by-Step}} = \mathcal{L}_{\text{label}} + \lambda_{\text{rat}} \mathcal{L}_{\text{rationale}}$.

Hsieh et al. [28] đã chứng minh một kết quả ấn tượng: Một mô hình học sinh nhỏ gọn kích thước **770 triệu tham số (770M)** khi được chắt lọc theo phương pháp Step-by-Step có thể đánh bại mô hình khổng lồ **PaLM-540B** trên nhiều bộ đo suy luận chuẩn, trong khi chỉ cần sử dụng một lượng dữ liệu huấn luyện bằng một phần nhỏ so với SFT thông thường! Nghiên cứu này là tiền đề trực tiếp giải thích tại sao các mô hình học sinh nhỏ như *DeepSeek-R1-Distill-Qwen-1.5B* lại sở hữu năng lực giải toán Olympic vượt trội hơn cả các mô hình 70B không qua chắt lọc suy luận [15].

Chương 6

Kiến trúc Bộ nhớ Hệ thống, Đánh giá Mô hình & Chẩn đoán Lỗi

Cầu nối Sư phạm (Từ Pre-training đến Post-training) 6.1. Chương cuối cùng của khóa học trang bị cho người học tư duy hệ thống toàn diện: từ việc giải phẫu từng byte bộ nhớ phần cứng VRAM (để hiểu tại sao việc huấn luyện đòi hỏi hạ tầng siêu máy tính), các kỹ thuật song song hóa (FSDP, ZeRO, LoRA), bản chất lý thuyết thông tin của Thuế Căn chỉnh (*Alignment Tax*), các mô hình thống kê khử thiên kiến trong đánh giá tự động (LLM-as-a-Judge, LC-AlpacaEval), và bảng chẩn đoán các hiện tượng bất thường khi huấn luyện.

6.1 Giải phẫu Bộ nhớ Phần cứng (VRAM Anatomy) từ Nguyên lý Gốc

Một ngộ nhận phổ biến của người mới tiếp cận Post-training là: *"Một mô hình 8 tỷ tham số (8B) lưu ở dạng nửa độ chính xác 16-bit (`bf16` tốn 2 bytes/tham số) chỉ cần* $8 \times 2 = 16$ *GB VRAM là có thể huấn luyện được!"*

Thực tế toán học và phần cứng hoàn toàn trái ngược: 16 GB chỉ đủ để nạp mô hình chạy **Suy luận (Inference)**. Khi bước vào **Huấn luyện (Training)** với bộ tối ưu hóa AdamW [39] và chế độ Độ chính xác Hỗn hợp (*Mixed Precision Training*) [42], bộ nhớ phần cứng bị chi phối bởi hai thành phần chính [49]:

6.1.1 1. Bộ nhớ Trạng thái Mô hình Tĩnh (Model States)

Với một mô hình có Ψ tham số (ví dụ $\Psi = 8 \times 10^9$ cho Llama-3-8B):

1. **Trọng số Mô hình (Weights W ở `bf16`):** Mỗi tham số tốn 2 bytes $\implies 2\Psi$ bytes (16 GB).
2. **Đạo hàm (Gradients $\nabla_{\mathbf{W}}$ ở `bf16`):** Mỗi tham số tốn 2 bytes $\implies 2\Psi$ bytes (16 GB).
3. **Bản sao Trọng số Gốc Độ chính xác Cao (Master Weights ở `fp32`):** Khi cập

nhật $\mathbf{W} \leftarrow \mathbf{W} - \eta \cdot \frac{m_t}{\sqrt{v_t}+\epsilon}$, bước nhảy thường có độ lớn cực nhỏ ($\approx 10^{-6}$). Trong định dạng 16-bit (`bf16`), các số quá nhỏ khi cộng vào số lớn sẽ bị hiện tượng triệt tiêu số học (*Underflow*). AdamW bắt buộc phải duy trì một bản sao chính xác tuyệt đối ở 32-bit: tốn 4Ψ bytes (32 GB).

4. **Trạng thái Động lượng Bậc một của AdamW (m_t ở `fp32`):** Tốn 4 bytes/tham số $\implies 4\Psi$ bytes (32 GB).

5. **Trạng thái Động lượng Bậc hai của AdamW (v_t ở `fp32`):** Tốn 4 bytes/tham số $\implies 4\Psi$ bytes (32 GB).

Định lý 6.1 (Tổng Bộ nhớ Trạng thái Mô hình Tĩnh)**.** *Tổng dung lượng VRAM tĩnh bắt buộc phải có cho một mô hình Ψ tham số huấn luyện bằng AdamW là:*

$$M_{static} = 2\Psi + 2\Psi + 4\Psi + 4\Psi + 4\Psi = \mathbf{16\Psi} \text{ bytes.} \tag{6.1}$$

*Với mô hình 8B tham số: $M_{static} = 16 \times 8 \times 10^9$ bytes $\approx$ **128** GB VRAM!*

6.1.2 2. Bộ nhớ Động: KV-Cache & Trạng thái Kích hoạt Trung gian (Activations)

Bên cạnh 128 GB bộ nhớ tĩnh, GPU còn phải chứa bộ nhớ động biến thiên theo kích thước lô (B) và độ dài chuỗi (L):

- **Bộ nhớ KV-Cache (trong pha Rollout sinh văn bản):** Với mô hình có n_{layers} tầng, $n_{\text{heads}}^{\text{KV}}$ đầu chú ý cho Key/Value, và chiều ẩn mỗi đầu d_{head}, bộ nhớ KV-Cache cho một chuỗi độ dài L là:

$$M_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{heads}}^{\text{KV}} \times d_{\text{head}} \times L \times B \times 2 \text{ bytes.} \tag{6.2}$$

  Kiến trúc **Grouped-Query Attention (GQA)** giúp giảm số đầu KV đi 4 đến 8 lần so với Multi-Head Attention truyền thống, tiết kiệm đáng kể VRAM trong pha sinh mẫu tự hồi quy.

- **Bộ nhớ Trạng thái Kích hoạt (Activations trong pha Training):** Để phục vụ việc tính đạo hàm trong pha lan truyền ngược, GPU bắt buộc phải lưu giữ toàn bộ các ma trận trung gian của pha lan truyền xuôi qua tất cả các tầng Transformer. Đặc biệt tại tầng cuối cùng (**Language Modeling Head**), tensor logits có kích thước:

$$\mathbf{Z}_{\text{logits}} \in \mathbb{R}^{B \times L \times |V|}. \tag{6.3}$$

  Với kích thước từ vựng hiện đại $|V| = 128,256$, độ dài chuỗi $L = 8,192$, và $B = 4$, chỉ riêng một tensor logits lưu ở định dạng `fp32` đã chiếm:

$$4 \times 8,192 \times 128,256 \times 4 \text{ bytes} \approx \mathbf{16.8} \text{ GB VRAM cho duy nhất một tensor!} \tag{6.4}$$

6.1.3 Cường độ Số học (Arithmetic Intensity) & Sự Nghẽn Cổ chai Phần cứng

Cường độ số học (*Arithmetic Intensity*) được định nghĩa là số phép tính dấu phẩy động (FLOPs) thực hiện trên mỗi byte dữ liệu truy xuất từ bộ nhớ:

$$I = \frac{\text{Số phép tính (FLOPs)}}{\text{Dung lượng đọc/ghi bộ nhớ (Bytes)}}. \tag{6.5}$$

- **Pha Rollout (Sinh token tự hồi quy):** Mỗi bước sinh 1 token chỉ thực hiện phép nhân ma trận–vector ($\approx 2\Psi$ FLOPs) nhưng phải đọc toàn bộ ma trận trọng số 2Ψ bytes từ bộ nhớ toàn cục (HBM). Cường độ số học đạt $I \approx \frac{2\Psi}{2\Psi} = \mathbf{1}$ FLOP/Byte. Vì các GPU hiện đại có năng lực tính toán hàng trăm TFLOPs nhưng băng thông bộ nhớ chỉ vài TB/s, pha Rollout bị giới hạn bởi băng thông bộ nhớ (**Memory-Bandwidth Bound**), nhân tính toán phần lớn thời gian phải ngồi chờ dữ liệu nạp vào.
- **Pha Training (Lan truyền xuôi và ngược):** Xử lý ma trận–ma trận trên toàn bộ chuỗi L, cường độ số học đạt $I \gg 100$ FLOP/Byte, hệ thống được khai thác tối đa công suất tính toán (**Compute-Bound**).

6.2 Bốn Trụ cột Tối ưu Hóa Hệ thống Post-Training Hiện đại

Để vận hành Post-training trên phần cứng thực tế, 4 kỹ thuật kiến trúc sống còn được áp dụng:

6.2.1 1. Tính trước Log-Probability Tham chiếu (Precomputing Reference Logprobs)

Trong Offline DPO (Chương 3), mô hình tham chiếu π_{ref} đóng băng hoàn toàn và tập dữ liệu (x, y_w, y_l) cố định. Thay vì nạp cả π_θ và π_{ref} lên GPU trong lúc huấn luyện (tốn gấp đôi bộ nhớ), ta chạy một lượt suy luận duy nhất trên tập dữ liệu để tính trước toàn bộ các số vô hướng $\log \pi_{\text{ref}}(y_w \mid x)$ và $\log \pi_{\text{ref}}(y_l \mid x)$, lưu vào ổ cứng rồi giải phóng hoàn toàn π_{ref} khỏi GPU!

6.2.2 2. Tái tính toán Kích hoạt (Activation / Gradient Checkpointing Chen et al. 11)

Thay vì lưu giữ toàn bộ kích hoạt trung gian của tất cả các tầng Transformer trong pha đi xuôi, ta chỉ lưu lại vector trạng thái ẩn tại ranh giới đầu mỗi tầng (*Checkpoint*) và xóa sạch các ma trận trung gian bên trong tầng đó. Khi pha lan truyền ngược lùi đến tầng nào, hệ thống sẽ tính xuôi lại các phép toán của riêng tầng đó.

- **Sự đánh đổi toán học:** Tăng thời gian tính toán thêm khoảng $25\% - 33\%$ (do phải tính xuôi 2 lần), nhưng **giảm bộ nhớ Activations từ $\mathcal{O}(N)$ xuống $\mathcal{O}(\sqrt{N})$ (tiết kiệm tới $70\% - 80\%$ bộ nhớ động)**, cho phép tăng chiều dài ngữ cảnh L lên gấp 4 lần.

6.2.3 3. Phân mảnh Trạng thái Mô hình (ZeRO-3 & FSDP Rajbhandari et al. 49, Zhao et al. 71)

Thay vì sao chép toàn bộ 128 GB trạng thái tĩnh của mô hình lên từng GPU (Data Parallelism truyền thống), kiến trúc **ZeRO-3 / FSDP (Fully Sharded Data Parallel)** chia nhỏ 16Ψ bytes trạng thái mô hình trải đều cho N_{GPU} thiết bị trong cụm:

$$M_{\text{per-GPU}} = \frac{2\Psi + 2\Psi + 12\Psi}{N_{\text{GPU}}} = \frac{16\Psi}{N_{\text{GPU}}}. \tag{6.6}$$

Khi tính toán đến tầng Transformer nào, các GPU thực hiện phép truyền thông `All-Gather` để gom tạm thời trọng số của tầng đó về, tính xong lập tức giải phóng ngay.

6.2.4 4. Thích nghi Hạng Thấp (LoRA & QLoRA Dettmers et al. 16, Hu et al. 29)

Dựa trên phát hiện của Aghajanyan et al. [2] rằng quá trình tinh chỉnh mô hình ngôn ngữ có **Số chiều Nội tại rất thấp (Low Intrinsic Dimension)**, phương pháp **LoRA (Low-Rank Adaptation)** [29] đóng băng hoàn toàn ma trận trọng số gốc $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ và chỉ huấn luyện một ma trận cập nhật hạng thấp:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}\mathbf{B}\mathbf{A}, \tag{6.7}$$

trong đó $\mathbf{B} \in \mathbb{R}^{d\times r}, \mathbf{A} \in \mathbb{R}^{r\times k}$ với hạng $r \ll \min(d, k)$ (thường chọn $r \in [8, 32]$), và α là hệ số tỷ lệ cố định.

- Vì số lượng tham số có thể huấn luyện giảm hơn 99%, toàn bộ gánh nặng 12Ψ bytes của bộ tối ưu AdamW gần như biến mất hoàn toàn!
- Trong **QLoRA** [16], ma trận gốc $\mathbf{W}_0$ còn được lượng tử hóa xuống định dạng 4-bit NormalFloat (NF4), cho phép tinh chỉnh mô hình 70B trên duy nhất một card đồ họa phổ thông 48 GB VRAM.

6.3 Thuế Căn chỉnh (Alignment Tax) & Hệ sinh thái Đánh giá Mô hình

6.3.1 Tại sao không thể dùng Perplexity? Khái niệm Thuế Căn chỉnh (Alignment Tax)

Trong Pre-training, chất lượng mô hình được đo bằng độ hỗn độn (**Perplexity – PPL**): $\text{PPL} = \exp\left(-\frac{1}{T}\sum_t \log P(x_t \mid x_{<t})\right)$. Tuy nhiên, **sau khi mô hình trải qua SFT, RLHF hoặc DPO, chỉ số Perplexity trên các tập văn bản Internet thông thường gần như chắc chắn sẽ tăng lên (xấu đi)!** Hiện tượng này được gọi là **Thuế Căn chỉnh (Alignment Tax)** [5, 43]:

- **Bản chất lý thuyết thông tin:** Base Model có entropy phân phối rất cao, bao phủ

mọi phong cách ngôn ngữ tồn tại trên mạng (từ văn chương, diễn đàn đến tin tức). Quá trình Post-training thực hiện **Thu hẹp Mốt Phân phối (Mode Collapse / Entropy Reduction)**: dồn phần lớn khối lượng xác suất vào một vùng không gian hẹp của phong cách trợ lý chuẩn mực, hữu ích và vô hại. Do đó, khả năng dự đoán các từ ngữ ngẫu nhiên trên mạng Internet của nó giảm sút, nhưng giá trị ứng dụng thực tiễn đối với con người lại tăng vọt.

6.3.2 Đánh giá Mở qua Giám khảo AI (LLM-as-a-Judge) & Ba Thiên kiến Nghiêm trọng

Đối với các câu hỏi mở (viết luận, tư vấn), phương pháp chuẩn mực hiện nay là sử dụng một mô hình giám khảo mạnh (như GPT-4) để so sánh câu trả lời của mô hình cần đo với một mô hình mốc qua hệ thống xếp hạng Elo / Bradley–Terry trên **Chatbot Arena** [12] và **MT-Bench** [72].

Tuy nhiên, nghiên cứu của Zheng et al. [72] và Dubois et al. [18] đã vạch rõ ba thiên kiến tâm lý học nguy hiểm của Giám khảo AI:

1. **Thiên kiến Vị trí (Position Bias):** Giám khảo AI có xu hướng ưu ái câu trả lời được đặt ở vị trí đầu tiên (Phương án A).
    - *Biện pháp khử chuẩn:* Luôn thực hiện đánh giá hai lượt với thứ tự hoán vị đảo ngược: Lượt 1 đưa (A, B), Lượt 2 đưa (B, A). Một mô hình chỉ được tính là thắng nếu chiến thắng ở cả hai lượt hoán vị.
2. **Thiên kiến Độ dài & Trình bày (Verbosity / Length Bias):** Giám khảo AI bị hấp dẫn bởi các câu trả lời dài dòng, chia nhiều đề mục in đậm bắt mắt ngay cả khi nội dung rỗng tuếch.
    - *Biện pháp khử chuẩn (**LC-AlpacaEval 2** Dubois et al. 18):* Sử dụng mô hình hồi quy tuyến tính tổng quát tách xác suất thắng của mô hình thành hai thành phần: năng lực nội tại và độ lệch chiều dài $\Delta L = |y_{\text{model}}| - |y_{\text{baseline}}|$, sau đó tính tỷ lệ thắng chuẩn hóa khi ép $\Delta L = 0$ (**Length-Controlled Win Rate**). Kỹ thuật này giúp độ tương quan giữa đánh giá tự động và đánh giá của con người trên Chatbot Arena tăng từ 0.94 lên 0.98!
3. **Thiên kiến Tự tôn (Self-Enhancement Bias):** Mô hình giám khảo thường chấm điểm thiên vị cho các mô hình có cùng họ kiến trúc hoặc được chắt lọc dữ liệu từ chính nó.

6.3.3 Các Bộ Tiêu chuẩn Đánh giá Kiểm chứng Khách quan (Verifiable Benchmarks)

Đối với các mô hình suy luận (Reasoning LLMs):

- **Toán học & Lập luận: GSM8K** [14] (Toán đố đa bước cấp 2), **MATH-500** [23] (Toán Olympic cấp trung học), và **AIME** (Kỳ thi chọn đội tuyển Olympic Toán học Hoa Kỳ).

- **Lập trình & Kỹ thuật Phần mềm: LiveCodeBench** [30] (Bộ đo thu thập liên tục các bài thi mới nhất từ LeetCode/Codeforces sau ngày đóng dữ liệu để chống rò rỉ đề thi) và **SWE-bench** [31] (Đánh giá khả năng tự động giải quyết các lỗi thực tế trên các kho mã nguồn mở GitHub).
- **Tuân thủ Ràng buộc Chỉ thị: IFEval** [74] (Kiểm tra bằng mã nguồn tự động khả năng tuân thủ các chỉ thị hình thức nghiêm ngặt, ví dụ: *"Viết bài văn gồm đúng 4 đoạn, đoạn 2 phải có ít nhất 3 dấu chấm phẩy và không được dùng chữ cái 'e' "*).

6.4 Bảng Chẩn đoán Lỗi Thực chiến cho Nghiên cứu Sinh Post-Training

Bảng 6.1 đúc kết các hiện tượng bất thường kinh điển trong quá trình huấn luyện Post-training, phân tích nguyên nhân gốc rễ toán học và giải pháp khắc phục chuẩn mực từ các phòng thí nghiệm hàng đầu.

Triệu chứng Huấn luyện	Nguyên nhân Gốc rễ Toán học	Giải pháp Khắc phục Lý thuyết
1. Sau SFT, mô hình tự lặp lại câu hỏi của người dùng hoặc nói lan man không dừng.	• Lỗi Prompt Masking: Quên gán $m_t = 0$ ở phần câu hỏi khiến mô hình học cách sinh lại câu hỏi; hoặc quên gán $m_t = 1$ tại token $\langle\vert\text{im_end}\vert\rangle$ của Assistant khiến mô hình không học được xác suất dừng.	• Kiểm tra lại dãy mặt nạ: chỉ bật $m_t = 1$ từ token đầu tiên của Assistant cho đến hết token kết thúc lượt $\langle\vert\text{im_end}\vert\rangle$ tương ứng (Chương 1).
2. Trong SFT đóng gói chuỗi, mô hình đột ngột đổi chủ đề giữa chừng khi đang trả lời.	• Lỗi rò rỉ ngữ cảnh: Dùng mặt nạ tam giác dưới thông thường của Pre-training thay vì Mặt nạ Đường chéo Khối; hoặc quên đặt lại vị trí RoPE về 0.	• Áp dụng Block-Diagonal Attention Mask cô lập tuyệt đối các mẫu hội thoại khác nhau và đặt lại dãy chỉ số vị trí RoPE về 0 tại đầu mỗi mẫu (Chương 1).
3. Trong DPO, phần thưởng ẩn tăng vọt lên trời chỉ sau 50 bước nhưng mô hình sinh ra từ lặp rác.	• Sụp đổ phân phối do mất neo KL: Hệ số β đặt quá nhỏ ($\beta \leq 0.001$) hoặc Learning Rate quá lớn ($\geq 10^{-4}$), làm đứt sợi dây liên kết với π_{ref}.	• Tăng hệ số β lên vùng chuẩn $[0.1, 0.5]$ và hạ Learning Rate của DPO xuống thấp hơn SFT 10 lần (thường 5×10^{-7} đến 5×10^{-6}).
4. Trong DPO, cả $\log \pi(y_w)$ và $\log \pi(y_l)$ đều tụt dốc âm sâu, văn bản xuất hiện ký tự lạ.	• Hiện tượng Dịch chuyển Xác suất (Likelihood Displacement): Đè xác suất câu dở kéo tụt luôn cả câu tốt do chia sẻ chung token.	• Áp dụng biến thể RPO / DPO+NLL: Cộng thêm thành phần mất mát SFT $\alpha\mathcal{L}_{\text{SFT}}(y_w)$ ($\alpha \approx 0.1$) để neo giữ xác suất câu đúng (Chương 3).
5. Trong GRPO, gradient biến mất ở nhiều bước liên tiếp, quá trình học bị đình trệ.	• Hiện tượng Đồng nhất Nhóm ($\text{std}(r) = 0$): Câu hỏi quá dễ (cả nhóm đều đúng) hoặc quá khó (cả nhóm đều sai), làm tử số $r_i - \bar{r} = 0 \implies A_i = 0$.	• Áp dụng Lọc Động (DAPO): Tự động loại bỏ các câu hỏi có toàn bộ nhóm đúng hết hoặc sai hết, chỉ giữ lại các câu hỏi có $0 < \bar{r} < 1$ (Chương 4).
6. Trong GRPO, các câu trả lời sai của mô hình ngày càng dài dòng vô tận.	• Thiên kiến độ dài do hệ số chia $\frac{1}{\lvert o_i \rvert}$: Khi $A_i < 0$, mô hình phát hiện kéo dài câu trả lời ra vô tận sẽ pha loãng mức phạt âm về 0.	• Áp dụng giải pháp Dr. GRPO: Thay thế hệ số $\frac{1}{\lvert o_i \rvert}$ bằng một hằng số chuẩn hóa cố định $L_{\max}$ trên toàn bộ tập dữ liệu (Chương 4).

Bảng 6.1: Bảng chẩn đoán và khắc phục các bất thường toán học trong huấn luyện Post-training.

Tài liệu tham khảo

[1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7319–7328, 2021.

[3] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[4] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238, pages 4447–4455, 2024.

[5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[6] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[7] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[8] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[11] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[12] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[13] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[14] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[15] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[16] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 10088–10115, 2023.

[17] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[18] Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. In *First Conference on Language Modeling (COLM)*, 2024.

[19] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235, pages 12690–12714, 2024.

[20] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, pages 10835–10866, 2023.

[21] Zorik Gekhman, Gal Yona, Roee Aharoni, Matan Eyal, Amir Feder, Roi Reichart, and Jonathan Herzig. Does fine-tuning LLMs on new knowledge encourage hallucinations? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7765–7784, 2024.

[22] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[23] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.

[24] John Hewitt. Initializing new word embeddings for pretrained language models. Stanford NLP Technical Note, https://nlp.stanford.edu/~johnhew/vocab-expansion.html, 2021.

[25] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[26] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[27] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11170–11189, 2024.

[28] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, 2023.

[29] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[30] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations (ICLR)*, 2025.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[32] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[33] Katie Kang, Eric Wallace, Christopher Tomlin, Aviral Kumar, and Sergey Levine. Unfamiliar finetuning examples control how language models hallucinate. *arXiv preprint arXiv:2403.05612*, 2024.

[34] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1317–1327, 2016.

[35] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[36] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback. In *International Conference on Machine Learning (ICML)*, 2024.

[37] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

[38] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[39] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[40] R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. John Wiley & Sons, 1959.

[41] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[42] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[43] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.

[44] Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddartha Naidu, and Colin White. Smaug: Fixing failure modes of preference optimisation with DPO-Positive. *arXiv preprint arXiv:2402.13228*, 2024.

[45] Richard Yuanzhe Pang, Weizhe Yuan, Kyunghyun Cho, He He, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[46] Ryan Park, Rafael Rafailov, Stefano Ermon, and Chelsea Finn. Disentangling length from quality in direct preference optimization. In *Findings of the Association for Computational Linguistics (ACL)*, pages 4998–5017, 2024.

[47] Robin L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 24(2):193–202, 1975.

[48] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 53728–53741, 2023.

[49] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16, 2020.

[50] Marc'Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.

[51] Noam Razin, Sadhika Malladi, Adithya Bhaskar, Danqi Chen, Sanjeev Arora, and Boris Hanin. Unintentional unalignment: Likelihood displacement in direct preference optimization. In *International Conference on Learning Representations (ICLR)*, 2025.

[52] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15, pages 627–635, 2011.

[53] John Schulman. Approximating KL divergence. Technical Note, `http://joschu.net/blog/kl-approx.html`, 2020.

[54] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[55] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[56] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Guo Daya. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[57] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025.

[58] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

[59] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 3008–3021, 2020.

[60] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[61] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[62] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 12, pages 1057–1063, 1999.

[63] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

[64] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[65] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[66] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022.

[67] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[68] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.

[69] Wei Xiong, Hanze Dong, Chenlu Ye, Ziqi Wang, Han Zhong, Heng Ji, Nan Jiang, and Tong Zhang. Iterative preference learning from human feedback: Bridging theory and practice for RLHF under KL-constraint. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[70] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[71] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.

[72] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46595–46623, 2023.

[73] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[74] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[75] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.